

Índice general

1. Representación de la información y estructura básica de un computador	3
1.1. Introducción	3
1.2. Función de un computador	4
1.3. Representación de la información	5
1.3.1. Sistema de numeración binario	5
1.3.2. Código binario	10
1.3.3. Codificación binaria de N elementos	11
1.3.4. Unidades de información	13
1.3.5. Código BCD	14
1.3.6. Sistema de numeración hexadecimal	17
1.3.7. Representación de números enteros	22
– Enteros sin signo	22
– Signo y magnitud	25
– Complemento a 1	28
– Complemento a 2	31
– Exceso a M	37
1.3.8. Representación de números reales	38
– Coma fija	39
– Coma flotante	44

Tema 1

Representación de la información y estructura básica de un computador

1.1. Introducción

Un **computador** es una máquina electrónica programable capaz de recibir, almacenar y procesar información, así como de proporcionar los resultados obtenidos, mediante la ejecución de programas informáticos. Los computadores estudiados en estos apuntes son **computadores digitales**, es decir, representan y procesan la información mediante valores discretos. En particular, los computadores digitales actuales utilizan fundamentalmente el sistema binario, basado en dos valores que se representan convencionalmente mediante los símbolos 0 y 1. Históricamente también se han desarrollado computadores analógicos, en los que la información se representa mediante magnitudes físicas continuas.

Un programa está constituido por un conjunto de instrucciones que especifican las operaciones que debe realizar el computador sobre unos determinados datos. Tanto las instrucciones como los datos se representan internamente mediante secuencias de símbolos binarios. El computador puede recibir datos a través de diferentes medios de entrada, procesarlos automáticamente bajo el control del programa en ejecución y proporcionar la información resultante a través de distintos medios de salida.

En la actualidad existe una gran variedad de computadores, con características muy diferentes en cuanto a tamaño, capacidad de procesamiento, capacidad de almacenamiento, consumo energético, coste y finalidad. Entre ellos se encuentran los **supercomputadores**, diseñados para realizar cálculos de gran complejidad y procesar enormes cantidades de información; los **servidores**, destinados principalmente a proporcionar servicios y recursos a otros computadores; los **computadores personales**, entre los que se incluyen los computadores de sobremesa y los portátiles; los **dispositivos móviles**, como los teléfonos inteligentes y las tabletas; y los **sistemas empuotrados**, que forman parte de otros dispositivos o sistemas y están diseñados habitualmente para realizar funciones específicas.

Los computadores desempeñan un papel fundamental en prácticamente todos los ámbitos de la sociedad actual. Se utilizan en actividades tan diversas como la investigación científica, la industria, las comunicaciones, el transporte, la medicina, la educación, la administración, el comercio y el ocio. Los computadores personales y los dispositivos móviles permiten realizar numerosas actividades cotidianas, mientras que los servidores proporcionan gran parte de los servicios accesibles a través de las redes de comunicaciones y los supercomputadores permiten abordar problemas científicos y técnicos que requieren una enorme capacidad de cálculo.

Además, muchos de los computadores que se utilizan diariamente no son identificados habitualmente como tales, ya que se encuentran integrados en sistemas de mayor tamaño. Automóviles, electrodomésticos, equipos médicos, sistemas industriales, dispositivos de comunicaciones y numerosos aparatos electrónicos incorporan sistemas empuotrados encargados de controlar y supervisar su funcionamiento. A pesar de las grandes diferencias existentes entre todos estos tipos de computadores, sus principios fundamentales de organización y funcionamiento presentan numerosos elementos comunes. El estudio de estos principios constituye el objetivo fundamental de esta primera parte de la asignatura.

En este tema se introducen los conceptos fundamentales necesarios para comprender la representación de la información y la estructura básica de un computador. En primer lugar, se estudia la función de un computador y se introducen los conceptos de dato, operación, instrucción y programa. A continuación, se analiza cómo se representa la información en un computador, estudiando los sistemas de numeración y los códigos utilizados para representar diferentes tipos de información, así como las principales unidades utilizadas para expresar cantidades de información y tiempos. Posteriormente, se presentan los fundamentos de la arquitectura de von Neumann y los componentes básicos que constituyen un computador. Finalmente, se estudian las estructuras que permiten la comunicación entre dichos componentes, prestando especial atención a la interconexión mediante buses.

1.2. Función de un computador

Para comprender cómo funciona un computador es necesario introducir previamente algunos conceptos fundamentales relacionados con la información que procesa y con las operaciones que realiza sobre ella.

La información que utiliza un computador se representa mediante **datos**. Un dato es una representación de información que puede ser almacenada y procesada por un computador. Los datos pueden representar elementos de naturaleza muy diversa, como números, caracteres, textos, imágenes, sonidos o vídeos. Aunque para el usuario estos elementos son muy diferentes, en el interior del computador todos ellos se representan mediante secuencias de símbolos binarios, como se estudiará en la sección siguiente.

El procesamiento de los datos consiste en la realización de determinadas **operaciones** sobre ellos. Estas operaciones pueden ser de muy distinta naturaleza. Por ejemplo, un computador puede sumar dos números, comparar dos valores, copiar un dato de un lugar a otro o determinar qué acción debe realizarse dependiendo del resultado de una operación anterior.

Una **instrucción** especifica una operación que el computador debe realizar y, cuando es necesario, los datos sobre los que debe efectuarse dicha operación. Por ejemplo, una instrucción puede indicar que se sumen dos números, que se compare un dato con otro o que se transfiera información de un lugar a otro. El conjunto de operaciones que puede especificarse mediante las instrucciones depende del computador considerado.

Un **programa** es una secuencia ordenada de instrucciones diseñada para realizar una determinada tarea. Las instrucciones especifican las operaciones que debe realizar el computador sobre unos determinados datos. Por tanto, durante la ejecución de un programa intervienen tanto las instrucciones que determinan las operaciones que deben realizarse como los datos que estas utilizan o generan.

La **ejecución de un programa** consiste en la obtención de las instrucciones que lo componen y en la realización de las operaciones especificadas por estas.

La **función básica de un computador** es la ejecución de programas. Mediante la ejecución de un programa, el computador procesa información de forma automática realizando una serie de operaciones sobre unos determinados datos.

La Figura 1.1 muestra de forma simplificada este funcionamiento: el computador recibe unos datos de entrada que procesa bajo el control de un programa y proporciona como resultado información de salida.

Desde un punto de vista más general, el funcionamiento de un computador puede describirse mediante cuatro funciones básicas: el almacenamiento de datos y programas, el procesamiento de datos, la transferencia de información entre el computador y su entorno y el control de todas estas operaciones. Estas funciones son realizadas por diferentes componentes del computador, que se estudiarán en la Sección ??.

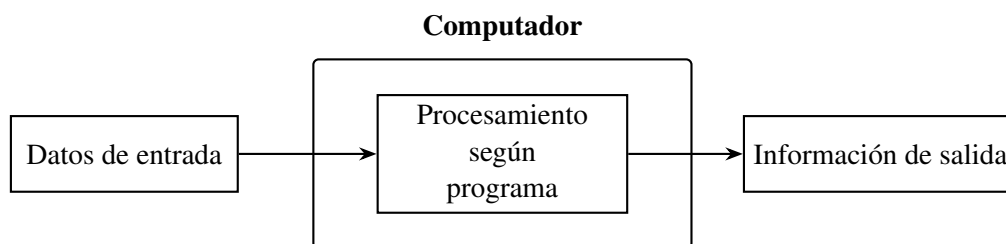


Figura 1.1 – Representación simplificada de la función de un computador.

1.3. Representación de la información

Un computador puede trabajar con información de naturaleza muy diversa: números, caracteres, textos, imágenes, sonidos, vídeos, instrucciones de un programa, etc. Sin embargo, los circuitos electrónicos que constituyen un computador digital trabajan con elementos que pueden encontrarse, de forma simplificada, en dos estados diferentes. Estos dos estados se representan convencionalmente mediante los símbolos 0 y 1.

La unidad elemental de información que puede tomar uno de estos dos valores recibe el nombre de **bit**, término procedente de la expresión inglesa *binary digit* (dígito binario). Por tanto, un bit puede tomar uno de los dos valores siguientes:

0 o 1

Para representar información más compleja se utilizan secuencias de bits. Por ejemplo, utilizando dos bits pueden formarse cuatro combinaciones:

00, 01, 10, 11

mientras que con tres bits pueden formarse ocho combinaciones:

000, 001, 010, 011, 100, 101, 110, 111

En general, mediante n bits pueden formarse 2^n combinaciones diferentes. Estas combinaciones pueden utilizarse para representar diferentes elementos de información, siempre que se establezca qué significado corresponde a cada una de ellas.

La **representación binaria de la información** consiste en representar la información mediante secuencias de bits, cuyos valores posibles son 0 y 1.

El empleo de los símbolos 0 y 1 puede tener significados diferentes dependiendo de la información que se desee representar. Cuando se utilizan como dígitos para representar números en base 2 se habla de **sistema de numeración binario**. En cambio, cuando se utilizan secuencias de bits para asignar una representación a los elementos de un determinado conjunto se habla de **código binario**.

Por ejemplo, la secuencia 1011 representa el número once cuando se interpreta como un número expresado en el sistema de numeración binario. Sin embargo, una secuencia de bits también puede utilizarse como código para representar un carácter, una instrucción o cualquier otro elemento de información. Por tanto, el significado de una secuencia de bits depende de la forma en que se interprete.

En las subsecciones siguientes se estudiarán los principales sistemas y códigos utilizados para representar la información en un computador.

1.3.1. Sistema de numeración binario

Antes de estudiar cómo se representan los números en un computador, conviene recordar brevemente los principales conjuntos numéricos. Los **números naturales** son los utilizados fundamentalmente para contar, como 1, 2, 3, etc.; los **números enteros** incluyen además el cero y los números negativos, como

-5 , 0 o 12 ; los **números racionales** son aquellos que pueden expresarse como cociente de dos números enteros, con denominador distinto de cero, como $2/3$, $-5/4$ o 7 ; y los **números irracionales** son los números reales que no pueden expresarse de esta forma, como $\sqrt{2}$ o π . El conjunto formado por los números racionales e irracionales constituye el conjunto de los **números reales**. Finalmente, los **números complejos** son aquellos que pueden expresarse en la forma $a + bi$, donde a y b son números reales e i es la unidad imaginaria, como $2 + 3i$.

En estos apuntes nos interesará principalmente la representación de números enteros y reales. A diferencia de lo que ocurre en matemáticas, en un computador se dispone de un número finito de bits para representar un número, lo que impone determinadas limitaciones a los valores que pueden representarse.

Para representar cantidades numéricas se utilizan **sistemas de numeración**. Un sistema de numeración establece un conjunto de símbolos, denominados **dígitos**, y unas reglas que permiten representar números mediante dichos símbolos.

Los sistemas de numeración que utilizaremos en estos apuntes son **sistemas de numeración posicionales**. En estos sistemas, el valor que representa un dígito depende tanto de su valor como de la posición que ocupa dentro del número. Cada sistema de numeración posicional se caracteriza por su **base**, que determina el número de dígitos diferentes que utiliza.

Cada posición tiene asociado un **peso**, que indica el valor correspondiente a dicha posición. En un sistema de numeración posicional de base b , los pesos son potencias de la base. La posición situada inmediatamente a la izquierda del punto tiene peso b^0 y, a medida que se avanza hacia la izquierda, los pesos son b^1 , b^2 , b^3 , etc. A la derecha del punto, los pesos son b^{-1} , b^{-2} , b^{-3} , etc. El valor representado por un número se obtiene sumando los productos de cada dígito por el peso correspondiente a su posición.

Por ejemplo, el **sistema de numeración decimal** es un sistema posicional de base 10 y utiliza los diez dígitos

$$0, 1, 2, 3, 4, 5, 6, 7, 8, 9$$

Los pesos de las posiciones son, por tanto, potencias de 10. Así, el número 325 puede expresarse como

$$325 = 3 \cdot 10^2 + 2 \cdot 10^1 + 5 \cdot 10^0$$

Si el número tiene parte fraccionaria, se utilizan también los pesos correspondientes a las posiciones situadas a la derecha del punto. Por ejemplo,

$$325.47 = 3 \cdot 10^2 + 2 \cdot 10^1 + 5 \cdot 10^0 + 4 \cdot 10^{-1} + 7 \cdot 10^{-2}$$

De forma general, en un sistema de numeración posicional de base b , un número con parte entera y parte fraccionaria

$$a_{n-1}a_{n-2} \dots a_1a_0.a_{-1}a_{-2} \dots a_{-m}$$

representa el valor

$$a_{n-1}b^{n-1} + \dots + a_1b + a_0 + a_{-1}b^{-1} + a_{-2}b^{-2} + \dots + a_{-m}b^{-m}$$

donde cada dígito a_i puede tomar uno de los valores comprendidos entre 0 y $b - 1$.

El signo $+$ o $-$ permite indicar si el número representado es positivo o negativo. Sin embargo, el signo no constituye un dígito del sistema de numeración. Más adelante se estudiará cómo se representan internamente en un computador los números negativos mediante secuencias de bits.

El **sistema de numeración binario** es un sistema de numeración posicional de base 2 que utiliza únicamente los dígitos 0 y 1. El peso de cada posición es una potencia de 2.

Para indicar la base en la que se encuentra expresado un número se puede escribir esta como subíndice. Por ejemplo,

$$(1011)_2$$

indica que 1011 está expresado en el sistema de numeración binario. Su valor en el sistema decimal se obtiene teniendo en cuenta el peso de cada una de sus posiciones:

$$(1011)_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 8 + 2 + 1 = (11)_{10}$$

En la Tabla 1.1 se muestran las representaciones binarias de los números decimales comprendidos entre 0 y 15. Todas ellas se muestran utilizando 4 bits para facilitar su comparación.

Decimal	Binario	Decimal	Binario
0	0000	8	1000
1	0001	9	1001
2	0010	10	1010
3	0011	11	1011
4	0100	12	1100
5	0101	13	1101
6	0110	14	1110
7	0111	15	1111

Tabla 1.1 – Representación binaria de los números decimales del 0 al 15.

► **Nota** En la Tabla 1.1 se han utilizado 4 bits para todas las representaciones con el fin de mostrarlas con la misma longitud. Obsérvese que los ceros situados a la izquierda no modifican el valor del número. Por ejemplo,

$$(0011)_2 = (11)_2 = (3)_{10}$$

Del mismo modo, las posiciones situadas a la derecha del punto tienen asociados pesos que son potencias de 2 con exponentes negativos. Por ejemplo,

$$(101.101)_2 = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3}$$

y, por tanto,

$$(101.101)_2 = (5.625)_{10}$$

Por tanto, los sistemas decimal y binario permiten representar cantidades numéricas utilizando diferentes bases y diferentes conjuntos de dígitos. Por ejemplo,

$$(11)_{10} = (1011)_2$$

► **Nota** El punto que separa la parte entera de la parte fraccionaria se utiliza aquí para expresar números en el sistema de numeración binario. Esto no significa que dicho punto tenga que almacenarse explícitamente en el computador. La forma en que se representan internamente los números con parte fraccionaria se estudiará posteriormente.

Como se ha visto, el peso asociado a cada bit depende de la posición que ocupa. En una secuencia de bits, el bit situado más a la izquierda se denomina **bit más significativo** (*Most Significant Bit*, *MSB*), mientras que el bit situado más a la derecha se denomina **bit menos significativo** (*Least Significant Bit*, *LSB*).

Por ejemplo, en el número binario $(1011)_2$, el bit más significativo es el situado más a la izquierda, que en este caso vale 1 y tiene asociado el mayor peso, 2^3 . El bit menos significativo es el situado más a

la derecha, que también vale 1 y tiene asociado el menor peso, 2^0 . Esta situación puede representarse de la siguiente forma:

$$\begin{array}{cccc} 1 & 0 & 1 & 1 \\ \text{MSB} & & & \text{LSB} \end{array}$$

Precisamente, estos pesos permiten obtener el valor decimal de un número expresado en el sistema binario. La conversión al sistema decimal puede realizarse directamente multiplicando cada dígito binario por la potencia de 2 correspondiente a su posición y sumando los resultados obtenidos.

El proceso inverso permite obtener la representación binaria de un número expresado en el sistema decimal. Si el número es entero y positivo, puede utilizarse el método de las divisiones sucesivas entre 2. Este método consiste en dividir sucesivamente el número y los cocientes obtenidos entre 2 hasta obtener un cociente igual a cero. Los restos de las divisiones, que necesariamente son 0 o 1, constituyen los dígitos de la representación binaria. Estos restos deben leerse en orden inverso al que han sido obtenidos.

● Ejemplo 1.1

Expresar el número $(45)_{10}$ en el sistema de numeración binario.

Solución

Se divide sucesivamente entre 2 el número y los cocientes que se van obteniendo, hasta obtener un cociente igual a cero:

$$\begin{array}{r|l} 45 & 2 \\ \hline 05 & 22 \\ 1 & \end{array} \quad \begin{array}{r|l} 22 & 2 \\ \hline 02 & 11 \\ 0 & \end{array} \quad \begin{array}{r|l} 11 & 2 \\ \hline 1 & 5 \end{array} \quad \begin{array}{r|l} 5 & 2 \\ \hline 1 & 2 \end{array} \quad \begin{array}{r|l} 2 & 2 \\ \hline 0 & 1 \end{array} \quad \begin{array}{r|l} 1 & 2 \\ \hline 1 & 0 \end{array}$$

Los restos obtenidos, en el orden en que se han calculado, son

$$1, 0, 1, 1, 0, 1$$

Para obtener la representación binaria deben leerse en orden inverso. Por tanto,

$$(45)_{10} = (101101)_2$$

El resultado puede comprobarse utilizando los pesos de las posiciones:

$$(101101)_2 = 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 32 + 8 + 4 + 1 = (45)_{10} \quad \bullet$$

Cuando el número decimal contiene una parte fraccionaria, esta puede convertirse al sistema binario mediante multiplicaciones sucesivas por 2. En cada multiplicación, la parte entera del resultado constituye un nuevo dígito binario. El proceso se repite utilizando la parte fraccionaria obtenida, y los dígitos binarios se toman en el mismo orden en que aparecen.

● Ejemplo 1.2

Expresar el número $(0.625)_{10}$ en el sistema de numeración binario.

Solución

Se multiplica sucesivamente por 2 la parte fraccionaria:

$$\begin{array}{rcl} 0.625 \cdot 2 & = & 1.25 \\ 0.25 \cdot 2 & = & 0.5 \\ 0.5 \cdot 2 & = & 1.0 \end{array}$$

Las partes enteras obtenidas, en el orden en que aparecen, son

$$1, 0, 1$$

Por tanto,

$$(0.625)_{10} = (0.101)_2$$

El resultado puede comprobarse utilizando los pesos de las posiciones:

$$(0.101)_2 = 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} = \frac{1}{2} + \frac{1}{8} = (0.625)_{10}$$

●

Si un número decimal tiene parte entera y parte fraccionaria, ambas partes se convierten por separado. La parte entera se convierte mediante divisiones sucesivas entre 2, mientras que la parte fraccionaria se convierte mediante multiplicaciones sucesivas por 2. Finalmente, las dos representaciones obtenidas se unen mediante el punto binario.

Por ejemplo, para convertir $(13.625)_{10}$ al sistema binario se convierten por separado la parte entera y la parte fraccionaria. Como

$$(13)_{10} = (1101)_2$$

y, según se ha visto anteriormente,

$$(0.625)_{10} = (0.101)_2$$

se obtiene

$$(13.625)_{10} = (1101.101)_2$$

El procedimiento de multiplicaciones sucesivas por 2 termina cuando la parte fraccionaria obtenida se hace igual a cero. Sin embargo, esto no ocurre para todos los números. Algunos números que tienen una representación decimal finita poseen una representación binaria con un número infinito de dígitos.

● Ejemplo 1.3

Expresar el número $(0.1)_{10}$ en el sistema de numeración binario.

Solución

Aplicando el método de las multiplicaciones sucesivas por 2:

$$\begin{array}{rcl} 0.1 \cdot 2 & = & 0.2 \\ 0.2 \cdot 2 & = & 0.4 \\ 0.4 \cdot 2 & = & 0.8 \\ 0.8 \cdot 2 & = & 1.6 \\ 0.6 \cdot 2 & = & 1.2 \\ 0.2 \cdot 2 & = & 0.4 \end{array}$$

A partir de este punto se repiten los mismos valores. En particular, vuelve a aparecer la parte fraccionaria 0.2, por lo que el proceso continuará repitiéndose indefinidamente. Los dígitos binarios obtenidos son

$$0, 0, 0, 1, 1, 0, 0, 1, 1, \dots$$

y, por tanto,

$$(0.1)_{10} = (0.0001100110011\dots)_2$$

Por tanto, el número $(0.1)_{10}$ no puede representarse mediante un número finito de dígitos binarios.

●

La razón por la que algunos números tienen una representación finita en un sistema de numeración y una representación infinita en otro está relacionada con la base utilizada. Una fracción, una vez simplificada, tiene una representación finita en una determinada base cuando los factores primos de su denominador son también factores de dicha base.

Por ejemplo,

$$0.1 = \frac{1}{10} = \frac{1}{2 \cdot 5}$$

tiene una representación finita en el sistema decimal porque la base 10 tiene como factores primos 2 y 5. Sin embargo, la base del sistema binario es 2, por lo que sus potencias solamente contienen el factor primo 2. Como el denominador de $1/10$ contiene también el factor 5, su representación binaria no puede ser finita.

En cambio,

$$0.625 = \frac{625}{1000} = \frac{5}{8} = \frac{5}{2^3}$$

sí tiene una representación binaria finita, ya que, una vez simplificada la fracción, su denominador es una potencia de 2. Como se ha visto anteriormente,

$$(0.625)_{10} = (0.101)_2$$

Por tanto, una fracción tiene una representación binaria finita si, una vez simplificada, su denominador es una potencia de 2. Por ejemplo, las fracciones $1/2$, $1/4$, $3/8$ y $5/16$ tienen representaciones binarias finitas, mientras que $1/3$, $1/5$ y $1/10$ tienen representaciones binarias infinitas.

Como un computador dispone de un número finito de bits para representar la información, los números cuya representación binaria requiere un número infinito de dígitos no pueden almacenarse de forma exacta utilizando un número finito de bits. En estos casos es necesario utilizar una aproximación. La forma en que los computadores representan internamente los números con parte fraccionaria se estudiará posteriormente.

► **Nota** Los procedimientos de conversión entre sistemas de numeración que se estudian en este tema permiten comprender cómo se relacionan las distintas representaciones de un mismo número y realizar las conversiones manualmente. Cuando se dispone de una calculadora o del software de cálculo matemático adecuado, estas conversiones pueden realizarse directamente. No obstante, conocer los procedimientos resulta también necesario para comprender y, en su caso, programar los algoritmos que permiten efectuar dichas conversiones.



1.3.2. Código binario

En la subsección anterior se ha estudiado el sistema de numeración binario, en el que los símbolos 0 y 1 se utilizan como dígitos para representar cantidades numéricas. Sin embargo, las secuencias de bits pueden utilizarse también para representar información que no tiene necesariamente un significado numérico.

Un **código** establece una correspondencia entre los elementos de un conjunto que se desea representar y un conjunto de símbolos o combinaciones de símbolos utilizados para representarlos. Por ejemplo, podría establecerse un código para representar los colores rojo, verde, azul y amarillo mediante las combinaciones 00, 01, 10 y 11.

Cuando los símbolos utilizados para realizar la codificación son los valores 0 y 1, se obtiene un **código binario**. En un código binario, cada elemento se representa mediante una determinada secuencia de bits denominada **palabra código** (*codeword*).

Un **código binario** es un sistema de representación en el que a cada elemento de un conjunto se le asigna una determinada secuencia de bits.

Por ejemplo, para representar cuatro colores diferentes podría utilizarse el código binario mostrado en la Tabla 1.2.

Nótese que la correspondencia entre los colores y las palabras código se ha establecido de forma arbitraria, por lo que podrían haberse asignado de otra manera las cuatro palabras código disponibles. En este ejemplo, la secuencia 10 representa el color azul, pero podría haberse elegido 00, 01 o 11 para representar este color.

Color	Palabra código
Rojo	00
Verde	01
Azul	10
Amarillo	11

Tabla 1.2 – Ejemplo de un código binario para representar cuatro colores.

El significado de una secuencia de bits depende, por tanto, de la forma en que se haya definido su interpretación. Por lo tanto, no debe confundirse un número expresado en el sistema de numeración binario con una palabra código de un código binario. En el primer caso, los bits son dígitos y sus posiciones tienen pesos que son potencias de 2. En el segundo, la secuencia de bits representa el elemento que le ha sido asignado mediante el código y no tiene necesariamente un significado numérico.

Por ejemplo, la secuencia 10, interpretada como un número binario, representa el número decimal 2, ya que

$$(10)_2 = 1 \cdot 2^1 + 0 \cdot 2^0 = (2)_{10}$$

Sin embargo, en el código binario de la Tabla 1.2, la palabra código 10 representa el color azul.

Los códigos binarios permiten representar mediante bits información de muy distinta naturaleza. Por ejemplo, pueden utilizarse para representar caracteres, instrucciones, estados de un sistema o diferentes opciones entre las que debe distinguir un computador.

1.3.3. Codificación binaria de N elementos

Un problema que aparece frecuentemente en diferentes aspectos relacionados con el hardware y el software de un computador consiste en determinar el número mínimo de bits necesarios para codificar un conjunto formado por N elementos diferentes.

Como cada bit puede tomar dos valores, 0 o 1, con un bit pueden formarse

$$2^1 = 2$$

palabras código diferentes; con dos bits,

$$2^2 = 4$$

con tres bits,

$$2^3 = 8$$

y, en general, con n bits pueden formarse

$$2^n$$

palabras código diferentes.

Por ejemplo, con 3 bits pueden formarse las ocho palabras código

$$000, \quad 001, \quad 010, \quad 011, \quad 100, \quad 101, \quad 110, \quad 111$$

por lo que pueden codificarse hasta ocho elementos diferentes.

Por tanto, para poder codificar N elementos diferentes mediante palabras código de n bits debe cumplirse

$$N \leq 2^n$$

Si se desea utilizar el menor número posible de bits, debe determinarse el menor número entero no negativo n que verifica esta desigualdad.

Una forma sencilla de determinar este número consiste en comparar N con potencias sucesivas de 2 hasta encontrar la primera que sea mayor o igual que N . En la Tabla 1.3 se muestran las primeras potencias de 2.

Potencia	Valor	Potencia	Valor
2^0	1	2^6	64
2^1	2	2^7	128
2^2	4	2^8	256
2^3	8	2^9	512
2^4	16	2^{10}	1 024
2^5	32		

Tabla 1.3 – Primeras potencias de 2.

► **Nota** Conviene familiarizarse con estas potencias de 2, ya que aparecen frecuentemente en diferentes aspectos relacionados con la organización y el funcionamiento de los computadores. ◀

El número mínimo de bits también puede calcularse directamente utilizando logaritmos. Para ello se utiliza la **función techo**, denotada por $\text{ceil}(x)$, que proporciona el menor número entero mayor o igual que x . Por ejemplo,

$$\text{ceil}(5.3) = 6 \quad \text{ceil}(7) = 7$$

El **número mínimo de bits** necesarios para codificar N elementos diferentes es el **menor** número entero no negativo n que verifica

$$N \leq 2^n \quad (1.1)$$

De forma equivalente, puede calcularse directamente mediante

$$n = \text{ceil}(\log_2 N) \quad (1.2)$$

Si N es una potencia de 2, es decir, si

$$N = 2^d$$

se necesitan exactamente d bits, ya que

$$n = \text{ceil}(\log_2 2^d) = \text{ceil}(d) = d$$

Cuando el número de elementos que se desea codificar coincide con el número de palabras código que pueden formarse con los bits utilizados, es decir, cuando

$$N = 2^n$$

pueden asignarse todas las palabras código disponibles. En cambio, si

$$N < 2^n$$

existen más palabras código disponibles que elementos que deben codificarse, por lo que algunas de ellas quedan sin asignar.

● Ejemplo 1.4

Determinar el número mínimo de bits necesarios para codificar:

- a) 512 elementos diferentes. b) 33 elementos diferentes.

Solución

a) Como

$$512 = 2^9$$

se necesitan 9 bits. Con estos 9 bits pueden formarse exactamente 512 palabras código, por lo que todas ellas pueden asignarse a los elementos que se desean codificar.

De forma equivalente, puede obtenerse el mismo resultado utilizando la función techo:

$$n = \text{ceil}(\log_2 512) = \text{ceil}(9) = 9$$

b) Como

$$2^5 = 32$$

con 5 bits no pueden codificarse los 33 elementos. Sin embargo,

$$2^6 = 64$$

por lo que con 6 bits pueden codificarse hasta 64 elementos. Por tanto, se necesitan 6 bits. De las 64 palabras código disponibles, se asignan 33 a los elementos que se desean codificar y quedan 31 sin asignar.

De forma equivalente, puede obtenerse el mismo resultado utilizando la función techo:

$$n = \text{ceil}(\log_2 33) = \text{ceil}(5.044 \dots) = 6$$

●

1.3.4. Unidades de información

Como se ha visto anteriormente, la información puede codificarse mediante secuencias de bits. Para expresar la cantidad de información que contiene una determinada representación se utilizan diferentes unidades.

La unidad básica es el **bit**, que corresponde a un dígito binario y puede tomar uno de los dos valores 0 y 1. Para representar el bit como unidad de información se utiliza el símbolo **b**.

Para expresar cantidades mayores de información resulta conveniente agrupar los bits. Un grupo de 8 bits recibe el nombre de **byte**. Para representar el byte como unidad de información se utiliza el símbolo **B**, por lo que

$$1 \text{ B} = 8 \text{ b}$$

Un byte puede contener $2^8 = 256$ combinaciones diferentes de bits.

Para expresar cantidades grandes de información se utilizan múltiplos del bit y del byte. Para ello pueden emplearse dos familias diferentes de prefijos: los **prefijos decimales**, basados en potencias de 10, de acuerdo con el Sistema Internacional de Unidades (SI), y los **prefijos binarios**, basados en potencias de 2, recogidos en estándares como IEEE 1541¹ e IEC 80000-13²

Los principales prefijos utilizados para expresar cantidades de información se muestran en la Tabla 1.4.

¹IEEE son las siglas de *Institute of Electrical and Electronics Engineers* (Instituto de Ingenieros Eléctricos y Electrónicos), organización profesional internacional que desarrolla, entre otras actividades, estándares técnicos en los ámbitos de la electricidad, la electrónica y la informática.

²IEC son las siglas de *International Electrotechnical Commission* (Comisión Electrotécnica Internacional), organización internacional que desarrolla estándares técnicos en los ámbitos de la electricidad, la electrónica y las tecnologías relacionadas.

Prefijos decimales			Prefijos binarios		
Prefijo	Símbolo	Valor	Prefijo	Símbolo	Valor
kilo	k	10^3	kibi	Ki	2^{10}
mega	M	10^6	mebi	Mi	2^{20}
giga	G	10^9	gibi	Gi	2^{30}
tera	T	10^{12}	tebi	Ti	2^{40}

Tabla 1.4 – Principales prefijos decimales y binarios utilizados para expresar cantidades de información

Los prefijos decimales se utilizan para representar múltiplos que son potencias de 10. Así, por ejemplo,

$$1 \text{ kB} = 10^3 \text{ B} = 1\,000 \text{ B}$$

$$1 \text{ MB} = 10^6 \text{ B} = 1\,000\,000 \text{ B}$$

$$1 \text{ GB} = 10^9 \text{ B} = 1\,000\,000\,000 \text{ B}$$

Por su parte, los prefijos binarios se utilizan para representar múltiplos que son potencias de 2. Así,

$$1 \text{ KiB} = 2^{10} \text{ B} = 1\,024 \text{ B}$$

$$1 \text{ MiB} = 2^{20} \text{ B} = 1\,048\,576 \text{ B}$$

$$1 \text{ GiB} = 2^{30} \text{ B} = 1\,073\,741\,824 \text{ B}$$

Es importante no confundir ambas familias de prefijos. Por ejemplo, 1 kB y 1 KiB no representan la misma cantidad de información:

$$1 \text{ kB} = 1\,000 \text{ B}, \quad 1 \text{ KiB} = 1\,024 \text{ B}$$

Aunque tradicionalmente se han utilizado en ocasiones los prefijos decimales kilo, mega o giga para hacer referencia a potencias de 2, en estos apuntes se distinguirán ambas notaciones: los prefijos k, M, G y T se utilizarán para las potencias de 10, mientras que Ki, Mi, Gi y Ti se utilizarán para las correspondientes potencias de 2.

Los prefijos anteriores pueden aplicarse tanto al bit como al byte. Por ejemplo,

$$1 \text{ Mb} = 10^6 \text{ b}, \quad 1 \text{ MB} = 10^6 \text{ B}$$

Como $1 \text{ B} = 8 \text{ b}$, se tiene

$$1 \text{ MB} = 8 \text{ Mb}$$

Por ello, es importante distinguir entre el símbolo **b**, que representa el bit, y el símbolo **B**, que representa el byte.

1.3.5. Código BCD

Un ejemplo de código binario utilizado para representar información numérica es el **código BCD**. Su nombre procede de la expresión inglesa *Binary Coded Decimal* (decimal codificado en binario). En este código, cada dígito decimal se representa de forma independiente mediante una palabra código de 4 bits.

Como el sistema decimal utiliza diez dígitos diferentes, del 0 al 9, es necesario disponer de al menos diez palabras código. Con 3 bits solamente pueden formarse

$$2^3 = 8$$

palabras código diferentes, mientras que con 4 bits pueden formarse

$$2^4 = 16$$

Por tanto, son necesarios 4 bits para codificar cada dígito decimal.

Existen diferentes códigos BCD, dependiendo de las palabras código de 4 bits que se asignen a los distintos dígitos decimales. Entre ellos se encuentran los códigos BCD 8421, 2421 y exceso a 3. Estos códigos utilizan palabras código de 4 bits para representar cada dígito decimal, pero difieren en la correspondencia establecida entre los dígitos decimales y dichas palabras código.

Uno de los más utilizados es el **código BCD 8421**. Se trata de un código ponderado en el que los cuatro bits tienen asociados, de izquierda a derecha, los pesos 8, 4, 2 y 1. El valor del dígito decimal representado se obtiene sumando los pesos correspondientes a los bits cuyo valor es 1. Por ejemplo,

$$0101 \longrightarrow 0 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 5$$

por lo que la palabra código 0101 representa el dígito decimal 5.

La denominación 8421 procede precisamente de los pesos asociados a las cuatro posiciones. Existen otros códigos BCD que utilizan una correspondencia diferente. Por ejemplo, el dígito decimal 5 se representa en algunos códigos BCD de la siguiente forma:

Código	Representación del dígito 5
BCD 8421	0101
BCD 2421	1011
BCD exceso a 3	1000

En estos apuntes se utilizará el código BCD 8421, por lo que, salvo que se indique lo contrario, cuando se haga referencia al código BCD se entenderá que se trata de este código. En la Tabla 1.5 se muestra la correspondencia entre los dígitos decimales y las palabras del código BCD 8421.

Dígito decimal	Código BCD 8421
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Tabla 1.5 – Representación de los dígitos decimales mediante el código BCD 8421.

Aunque con 4 bits pueden formarse 16 palabras código diferentes, el código BCD 8421 solamente utiliza diez de ellas para representar los dígitos decimales. Por tanto, las seis palabras código restantes,

$$1010, \quad 1011, \quad 1100, \quad 1101, \quad 1110, \quad 1111$$

no se utilizan para representar dígitos decimales en este código.

Para representar mediante BCD un número formado por varios dígitos decimales, se codifica cada uno de sus dígitos por separado. Por ejemplo, para representar el número decimal 59, se codifican independientemente los dígitos 5 y 9. Según la Tabla 1.5,

$$5 \longrightarrow 0101 \quad 9 \longrightarrow 1001$$

por lo que

$$(59)_{10} \longrightarrow 0101\ 1001 \quad (\text{BCD})$$

Es importante observar que la representación BCD de un número decimal **no coincide**, en general, con su representación en el sistema de numeración binario. En BCD se codifica por separado cada dígito decimal mediante una palabra código de 4 bits, mientras que en el sistema binario se representa directamente el valor del número utilizando potencias de 2. Por ejemplo, si el número decimal 59 se convierte directamente al sistema binario se obtiene

$$(59)_{10} = (111011)_2$$

mientras que, como se acaba de obtener, su representación mediante el código BCD 8421 es

$$0101\ 1001$$

El código BCD facilita el tratamiento de información expresada mediante dígitos decimales, ya que mantiene una correspondencia directa entre cada dígito decimal y su representación binaria. Sin embargo, utiliza más bits que la representación directa de un número en el sistema binario. Por ejemplo, el número decimal 59 necesita 8 bits en BCD, mientras que su representación binaria 111011 necesita solamente 6 bits.

A pesar de este mayor número de bits, el código BCD resulta útil en aplicaciones en las que se trabaja fundamentalmente con dígitos decimales y es importante realizar de forma sencilla la conversión entre su representación interna y su presentación decimal. Por este motivo, se ha utilizado, entre otras aplicaciones, en calculadoras, sistemas de visualización numérica y determinados sistemas de procesamiento de información decimal.

● Ejemplo 1.5

Realizar las siguientes conversiones:

- a) Convertir $(275)_{10}$ a código BCD 8421. b) Convertir $(101101)_2$ a código BCD 8421.

Solución

- a) Se codifica por separado cada uno de los dígitos decimales:

$$2 \longrightarrow 0010, \quad 7 \longrightarrow 0111, \quad 5 \longrightarrow 0101$$

Por tanto,

$$(275)_{10} \longrightarrow 0010\ 0111\ 0101 \quad (\text{BCD})$$

- b) En primer lugar, se convierte el número binario al sistema decimal:

$$(101101)_2 = 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = (45)_{10}$$

A continuación, se codifican por separado los dígitos decimales 4 y 5:

$$4 \longrightarrow 0100 \quad 5 \longrightarrow 0101$$

Por tanto,

$$(101101)_2 \longrightarrow (45)_{10} \longrightarrow 0100\ 0101 \quad (\text{BCD})$$

●

1.3.6. Sistema de numeración hexadecimal

El **sistema de numeración hexadecimal** es un sistema de numeración posicional de base 16. Por tanto, necesita 16 dígitos diferentes para representar los valores comprendidos entre 0 y 15. Los diez primeros se representan mediante los dígitos habituales del sistema decimal, mientras que para los seis restantes se utilizan las letras A, B, C, D, E y F:

$$0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F$$

Las letras utilizadas como dígitos hexadecimales representan los siguientes valores decimales:

$$A = 10, \quad B = 11, \quad C = 12, \quad D = 13, \quad E = 14, \quad F = 15$$

Al tratarse de un sistema posicional de base 16, el peso de cada posición es una potencia de 16. Así, la posición situada inmediatamente a la izquierda del punto tiene peso 16^0 y, a medida que se avanza hacia la izquierda, los pesos son 16^1 , 16^2 , 16^3 , etc. A la derecha del punto, los pesos son 16^{-1} , 16^{-2} , 16^{-3} , etc.

Para indicar que un número se encuentra expresado en el sistema hexadecimal se utiliza el subíndice 16. Por ejemplo,

$$(2A7)_{16}$$

representa el número cuyo valor decimal es

$$(2A7)_{16} = 2 \cdot 16^2 + 10 \cdot 16^1 + 7 \cdot 16^0 = 512 + 160 + 7 = (679)_{10}$$

La conversión de un número entero expresado en el sistema decimal al sistema hexadecimal puede realizarse mediante divisiones sucesivas entre 16, de forma análoga al procedimiento utilizado para convertir un número decimal al sistema binario. Los restos obtenidos, leídos en orden inverso, constituyen los dígitos de la representación hexadecimal. Cuando un resto tiene un valor comprendido entre 10 y 15, debe sustituirse por la letra hexadecimal correspondiente.

● Ejemplo 1.6

Expresar el número $(725)_{10}$ en el sistema de numeración hexadecimal.

Solución

Se realizan divisiones sucesivas entre 16 hasta obtener un cociente igual a 0:

$$\begin{array}{r|l} 725 & 16 \\ 85 & 45 \\ 5 & \end{array} \quad \begin{array}{r|l} 45 & 16 \\ 13 & 2 \end{array} \quad \begin{array}{r|l} 2 & 16 \\ 2 & 0 \end{array}$$

Los restos obtenidos, en el orden en que se han calculado, son

$$5, 13, 2$$

Como el valor 13 se representa mediante el dígito hexadecimal D, al leer los restos en orden inverso se obtiene

$$(725)_{10} = (2D5)_{16}$$

El resultado puede comprobarse utilizando los pesos de las posiciones:

$$(2D5)_{16} = 2 \cdot 16^2 + 13 \cdot 16^1 + 5 \cdot 16^0 = 512 + 208 + 5 = (725)_{10}$$



El sistema hexadecimal resulta especialmente útil en informática debido a su relación directa con el sistema binario. Como

$$16 = 2^4$$

cada dígito hexadecimal puede representarse exactamente mediante un grupo de 4 bits. La correspondencia entre los dígitos hexadecimales y los grupos de 4 bits se muestra en la Tabla 1.6.

Hexadecimal	Binario	Hexadecimal	Binario
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

Tabla 1.6 – Correspondencia entre los dígitos hexadecimales y los grupos de 4 bits.

Esta correspondencia permite realizar directamente conversiones entre los sistemas binario y hexadecimal, sin necesidad de pasar por el sistema decimal. Para convertir un número binario al sistema hexadecimal se agrupan sus bits de cuatro en cuatro comenzando por el punto binario. En la parte entera, los grupos se forman hacia la izquierda y, en la parte fraccionaria, hacia la derecha.

Por ejemplo, para convertir el número

$$(1011011110)_2$$

al sistema hexadecimal, se agrupan sus bits de cuatro en cuatro comenzando por la derecha:

$$(10 \ 1101 \ 1110)_2$$

Como el grupo situado más a la izquierda contiene solamente dos bits, se añaden dos ceros a su izquierda para completar un grupo de cuatro:

$$(0010 \ 1101 \ 1110)_2$$

Utilizando la correspondencia de la Tabla 1.6,

$$0010 \rightarrow 2, \quad 1101 \rightarrow D, \quad 1110 \rightarrow E$$

por lo que

$$(1011011110)_2 = (2DE)_{16}$$

El procedimiento inverso es igualmente directo. Para convertir un número hexadecimal al sistema binario basta con sustituir cada dígito hexadecimal por el grupo de 4 bits correspondiente. Por ejemplo,

$$(3A7)_{16}$$

se convierte de la siguiente forma:

$$3 \rightarrow 0011, \quad A \rightarrow 1010, \quad 7 \rightarrow 0111$$

y, por tanto,

$$(3A7)_{16} = (0011 \ 1010 \ 0111)_2$$

Los ceros situados a la izquierda de la representación binaria pueden eliminarse sin modificar el valor representado, por lo que también puede escribirse

$$(3A7)_{16} = (1110100111)_2$$

► Nota

La conversión de un número decimal al sistema hexadecimal también puede realizarse utilizando el sistema binario como representación intermedia. En primer lugar, se convierte el número decimal al sistema binario mediante divisiones sucesivas entre 2 y, posteriormente, se agrupan los bits de cuatro en cuatro para obtener directamente los dígitos hexadecimales.

Por ejemplo,

$$(725)_{10} = (1011010101)_2$$

y, agrupando los bits de cuatro en cuatro desde la derecha,

$$(0010\ 1101\ 0101)_2 = (2D5)_{16}$$

Este procedimiento puede resultar cómodo al realizar la conversión manualmente, ya que las divisiones entre 2 son especialmente sencillas.

El sistema hexadecimal permite expresar de forma mucho más compacta secuencias binarias largas. Por esta razón se utiliza frecuentemente en informática para representar, entre otros elementos, direcciones de memoria, contenidos de registros y valores binarios que resultarían poco manejables si se escribieran directamente como secuencias de bits.

● Ejemplo 1.7

Realizar las siguientes conversiones:

- a) $(3A5C)_{16}$ al sistema decimal.
- b) $(12500)_{10}$ al sistema hexadecimal.
- c) $(5D2A)_{16}$ al sistema binario.
- d) $(1011011000111110)_2$ al sistema hexadecimal.

Solución

- a) Para convertir $(3A5C)_{16}$ al sistema decimal, se utilizan los pesos de las posiciones:

$$\begin{aligned}(3A5C)_{16} &= 3 \cdot 16^3 + 10 \cdot 16^2 + 5 \cdot 16^1 + 12 \cdot 16^0 \\ &= 12288 + 2560 + 80 + 12 = (14940)_{10}\end{aligned}$$

- b) La conversión de $(12500)_{10}$ al sistema hexadecimal se realiza mediante divisiones sucesivas entre 16 hasta obtener un cociente igual a 0:

$$\begin{array}{r|l|l|l|l|l} 12500 & 16781 & 16 & 48 & 16 & 3 \\ 130 & 78141 & 48 & 0 & 3 & 3 \\ 20 & 13 & & & & \\ 4 & & & & & \end{array}$$

Los restos obtenidos, en el orden en que se han calculado, son

$$4, 13, 0, 3$$

Como el valor 13 corresponde al dígito hexadecimal D, al leer los restos en orden inverso se obtiene

$$(12500)_{10} = (30D4)_{16}$$

- c) Para obtener la representación binaria de $(5D2A)_{16}$, se sustituye cada dígito hexadecimal por el grupo de 4 bits correspondiente:

$$5 \rightarrow 0101, \quad D \rightarrow 1101, \quad 2 \rightarrow 0010, \quad A \rightarrow 1010$$

Por tanto,

$$(5D2A)_{16} = (0101\ 1101\ 0010\ 1010)_2$$

- d) En la conversión de $(1011011000111110)_2$ al sistema hexadecimal, se agrupan los bits de cuatro en cuatro comenzando por la derecha:

$$(1011\ 0110\ 0011\ 1110)_2$$

A continuación, se sustituye cada grupo por el dígito hexadecimal correspondiente:

$$1011 \rightarrow B, \quad 0110 \rightarrow 6, \quad 0011 \rightarrow 3, \quad 1110 \rightarrow E$$

Por tanto,

$$(1011011000111110)_2 = (B63E)_{16}$$

● Ejemplo 1.8

Realizar las siguientes conversiones:

- a) $(12A5)_{16}$ a código BCD. b) 0010 0101 0110 1001 en BCD al sistema hexadecimal.

Solución

- a) Para expresar $(12A5)_{16}$ mediante el código BCD, primero se convierte el número hexadecimal al sistema decimal:

$$\begin{aligned} (12A5)_{16} &= 1 \cdot 16^3 + 2 \cdot 16^2 + 10 \cdot 16^1 + 5 \cdot 16^0 \\ &= 4096 + 512 + 160 + 5 = (4773)_{10} \end{aligned}$$

A continuación, se codifica por separado cada uno de los dígitos decimales mediante BCD:

$$4 \rightarrow 0100, \quad 7 \rightarrow 0111, \quad 7 \rightarrow 0111, \quad 3 \rightarrow 0011$$

Por tanto,

$$(12A5)_{16} \rightarrow (4773)_{10} \rightarrow 0100\ 0111\ 0111\ 0011 \quad (\text{BCD})$$

- b) Para convertir 0010 0101 0110 1001 en BCD al sistema hexadecimal, se obtiene primero el número decimal representado por el código BCD. Para ello, se interpreta cada grupo de 4 bits como un dígito decimal:

$$0010 \rightarrow 2, \quad 0101 \rightarrow 5, \quad 0110 \rightarrow 6, \quad 1001 \rightarrow 9$$

Por tanto,

$$0010\ 0101\ 0110\ 1001 \quad (\text{BCD}) \rightarrow (2569)_{10}$$

A continuación, se convierte $(2569)_{10}$ al sistema hexadecimal mediante divisiones sucesivas entre 16:

$$\begin{array}{r|l} 2569 & 16 \\ \hline 96 & 160 \\ 9 & 9 \end{array} \quad \begin{array}{r|l} 160 & 16 \\ \hline 0 & 10 \end{array} \quad \begin{array}{r|l} 10 & 16 \\ \hline 10 & 0 \end{array}$$

Los restos obtenidos, en el orden en que se han calculado, son

$$9, 0, 10$$

Como el valor 10 corresponde al dígito hexadecimal A, al leer los restos en orden inverso se obtiene

$$(2569)_{10} = (A09)_{16}$$

Por tanto,

$$0010\ 0101\ 0110\ 1001 \quad (\text{BCD}) \longrightarrow (A09)_{16}$$

●

Otro sistema de numeración utilizado en informática es el **sistema de numeración octal**, que es un sistema de numeración posicional de base 8 y utiliza los dígitos del 0 al 7. Como

$$8 = 2^3$$

cada dígito octal puede representarse exactamente mediante un grupo de 3 bits. Por tanto, la conversión entre los sistemas binario y octal puede realizarse directamente agrupando o sustituyendo los bits de tres en tres.

● Ejemplo 1.9

Realizar las siguientes conversiones:

a) $(110101011)_2$ al sistema octal.

b) $(527)_8$ al sistema binario.

Solución

a) Para convertir $(110101011)_2$ al sistema octal, se agrupan los bits de tres en tres comenzando por la derecha:

$$(110\ 101\ 011)_2$$

A continuación, se sustituye cada grupo por el dígito octal correspondiente:

$$110 \longrightarrow 6, \quad 101 \longrightarrow 5, \quad 011 \longrightarrow 3$$

Por tanto,

$$(110101011)_2 = (653)_8$$

b) Para obtener la representación binaria de $(527)_8$, se sustituye cada dígito octal por el grupo de 3 bits correspondiente:

$$5 \longrightarrow 101, \quad 2 \longrightarrow 010, \quad 7 \longrightarrow 111$$

Por tanto,

$$(527)_8 = (101\ 010\ 111)_2$$

●

El sistema octal también permite expresar de forma compacta secuencias binarias, aunque actualmente se utiliza con menor frecuencia que el sistema hexadecimal.

1.3.7. Representación de números enteros

Para representar números enteros en un computador mediante un número determinado de bits es necesario establecer qué valor entero corresponde a cada una de las posibles secuencias de bits. Como se ha visto anteriormente, con n bits pueden formarse 2^n secuencias diferentes y, por tanto, pueden representarse como máximo 2^n valores diferentes.

La correspondencia entre estas secuencias de bits y los números enteros depende del sistema de representación utilizado. Si solamente se desean representar enteros no negativos, puede utilizarse directamente el sistema de numeración binario. En cambio, si se desean representar también números negativos, es necesario establecer un procedimiento que permita distinguir los valores positivos de los negativos.

A continuación se estudiarán diferentes sistemas de representación de números enteros: enteros sin signo, signo y magnitud, complemento a 1, complemento a 2 y exceso a M .

Enteros sin signo

La **representación de enteros sin signo** es un sistema de representación de números enteros en el que todos los bits disponibles se utilizan para expresar directamente el valor del número en el sistema binario.

Si se utilizan n bits, pueden representarse 2^n valores diferentes. La menor representación posible está formada por todos los bits a 0,

$$\underbrace{00 \dots 0}_{n \text{ bits}}$$

y representa el valor 0. La mayor está formada por todos los bits a 1,

$$\underbrace{11 \dots 1}_{n \text{ bits}}$$

y representa el valor

$$2^{n-1} + 2^{n-2} + \dots + 2^1 + 2^0 = 2^n - 1$$

Con n bits, la **representación de enteros sin signo** permite representar los valores comprendidos entre

$$0 \leq x \leq 2^n - 1$$
(1.3)

Por ejemplo, con 8 bits pueden representarse

$$2^8 = 256$$

valores diferentes, comprendidos entre

$$0 \quad \text{y} \quad 2^8 - 1 = 255$$

Algunas de las representaciones posibles con 8 bits y sus correspondientes valores son:

● Ejemplo 1.10

Se dispone de 5 bits para representar enteros sin signo. ¿Cuántos valores diferentes pueden representarse? ¿Cuáles son el menor y el mayor valor representables?

Solución

Con 5 bits pueden formarse

$$2^5 = 32$$

secuencias diferentes y, por tanto, pueden representarse 32 valores diferentes.

Representación	Valor decimal
00000000	0
00000001	1
00000010	2
00000011	3
⋮	⋮
11111100	252
11111101	253
11111110	254
11111111	255

El menor valor representable es

$$0$$

y el mayor es

$$2^5 - 1 = 31$$

Por tanto, con 5 bits pueden representarse 32 valores enteros diferentes, comprendidos entre 0 y 31.

● Ejemplo 1.11

Se desean representar conjuntamente, como enteros sin signo y utilizando el mismo número de bits, los números 511, 1024 y 256. Determinar el número mínimo de bits necesario.

Solución

Para determinar el número mínimo de bits necesario, debe calcularse cuántos bits se necesitan para representar cada uno de los números. Como todos ellos deben representarse utilizando el mismo número de bits, habrá que utilizar un número de bits suficiente para representar el mayor de ellos.

Para representar el número 511, se tiene

$$2^8 - 1 = 255 < 511$$

por lo que 8 bits no son suficientes. Sin embargo,

$$2^9 - 1 = 511$$

por lo que son necesarios, como mínimo, 9 bits.

Para representar el número 1024, se tiene

$$2^{10} - 1 = 1023 < 1024$$

por lo que 10 bits no son suficientes. Sin embargo,

$$2^{11} - 1 = 2047 > 1024$$

por lo que son necesarios, como mínimo, 11 bits.

Finalmente, para representar el número 256, se tiene

$$2^8 - 1 = 255 < 256$$

por lo que 8 bits no son suficientes. Sin embargo,

$$2^9 - 1 = 511 > 256$$

por lo que son necesarios, como mínimo, 9 bits.

Por tanto, para poder representar conjuntamente los tres números utilizando el mismo número de bits, son necesarios, como mínimo, 11 bits.

► **Nota** No debe confundirse el número de bits necesarios para **representar un número entero** con el número de bits necesarios para **codificar un conjunto de elementos**. En el ejemplo anterior se han considerado tres números, 511, 1024 y 256, pero no se pretende asignar una palabra código diferente a cada uno de ellos, sino representar el valor numérico de cada uno mediante una secuencia de bits.

Si únicamente se quisieran codificar tres elementos diferentes, independientemente de su valor numérico, serían suficientes 2 bits, ya que con ellos pueden formarse $2^2 = 4$ palabras código diferentes. En cambio, para representar los valores numéricos 511, 1024 y 256 como enteros sin signo se necesitan, respectivamente, 9, 11 y 9 bits.

La conversión entre la representación de un entero sin signo y su valor decimal se realiza de la misma forma que la conversión entre los sistemas de numeración binario y decimal estudiada anteriormente. La única particularidad es que la representación debe ajustarse al número de bits establecido, añadiendo ceros por la izquierda cuando sea necesario.

● Ejemplo 1.12

Realizar las siguientes conversiones utilizando una representación de enteros sin signo de 8 bits:

a) Representar el número 45

b) Determinar el valor decimal de 1 0 1 1 0 1 1 0

Solución

a) Para representar el número 45, se convierte primero al sistema binario:

$$(45)_{10} = (101101)_2$$

Como la representación debe realizarse utilizando 8 bits, se añaden ceros por la izquierda hasta completar los 8 bits:

$$45 \longrightarrow 00101101$$

b) Para determinar el valor decimal representado por

$$10110110$$

se interpreta la secuencia como un número binario:

$$1 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 182$$

Por tanto,

$$10110110 \longrightarrow 182$$

Signo y magnitud

La representación de enteros sin signo solamente permite representar valores no negativos. Para representar también números enteros negativos es necesario establecer algún procedimiento que permita indicar el signo del número.

La **representación en signo y magnitud** es un sistema de representación de números enteros en el que el bit más significativo, es decir, el situado más a la izquierda, se utiliza para indicar el signo del número y recibe el nombre de **bit de signo**. Los restantes bits se utilizan para representar, en el sistema binario, el valor absoluto del número.

Para el bit de signo se adopta la siguiente convención:

$$0 \rightarrow \text{positivo} \quad 1 \rightarrow \text{negativo}$$

Por tanto, si se utilizan n bits para representar un número entero, se dispone de un bit para indicar el signo y de $n - 1$ bits para representar su valor absoluto.

Por ejemplo, utilizando 8 bits, el número +5 se representa como

$$+5 \rightarrow \underbrace{0}_{\text{signo}} \underbrace{0000101}_{\text{magnitud}}$$

mientras que el número -5 se representa como

$$-5 \rightarrow \underbrace{1}_{\text{signo}} \underbrace{0000101}_{\text{magnitud}}$$

Como se dispone de $n - 1$ bits para representar la magnitud, el mayor valor absoluto que puede representarse es

$$2^{n-1} - 1$$

Por tanto, los valores representables están comprendidos entre

$$-(2^{n-1} - 1) \quad \text{y} \quad 2^{n-1} - 1$$

Con n bits, la **representación en signo y magnitud** permite representar los valores comprendidos entre

$$-(2^{n-1} - 1) \leq x \leq 2^{n-1} - 1 \quad (1.4)$$

● Ejemplo 1.13

Se dispone de 6 bits para representar números enteros en signo y magnitud. ¿Cuántos valores enteros diferentes pueden representarse? ¿Cuáles son el menor y el mayor valor representables?

Solución

En la representación en signo y magnitud, uno de los bits se utiliza para indicar el signo del número. Por tanto, de los 6 bits disponibles, quedan 5 bits para representar la magnitud.

Con 5 bits, el mayor valor que puede representarse como magnitud es

$$2^5 - 1 = 31$$

por lo que pueden representarse los valores comprendidos entre -31 y 31.

Aunque con 6 bits pueden formarse

$$2^6 = 64$$

secuencias diferentes, dos de ellas representan el valor cero, una con signo positivo y otra con signo negativo. Por tanto, el número de valores enteros diferentes que pueden representarse es

$$64 - 1 = 63$$

En consecuencia, con 6 bits pueden representarse 63 valores enteros diferentes, comprendidos entre -31 y 31 .

● Ejemplo 1.14

Se desean representar conjuntamente, en signo y magnitud y utilizando el mismo número de bits, los números -127 , 63 y -128 . Determinar el número mínimo de bits necesario.

Solución

En la representación en signo y magnitud con n bits, uno de ellos se utiliza para representar el signo y los $n - 1$ restantes para representar la magnitud. Por tanto, el mayor valor absoluto que puede representarse es

$$2^{n-1} - 1$$

Para representar el número -127 , su magnitud es 127 . Con 7 bits para la magnitud se puede representar hasta

$$2^7 - 1 = 127$$

por lo que, teniendo en cuenta el bit de signo, son necesarios 8 bits.

Para representar el número 63 , su magnitud es 63 . Con 6 bits para la magnitud se puede representar hasta

$$2^6 - 1 = 63$$

por lo que, teniendo en cuenta el bit de signo, son necesarios 7 bits.

Finalmente, para representar el número -128 , su magnitud es 128 . Con 7 bits para la magnitud solamente puede representarse hasta

$$2^7 - 1 = 127 < 128$$

por lo que 8 bits no son suficientes. Con 8 bits para la magnitud se puede representar hasta

$$2^8 - 1 = 255$$

y, teniendo en cuenta el bit de signo, son necesarios 9 bits.

Por tanto, para poder representar conjuntamente los tres números en signo y magnitud utilizando el mismo número de bits, son necesarios, como mínimo, 9 bits.

Para obtener la representación en signo y magnitud de un número entero, se representa en binario su valor absoluto utilizando los bits destinados a la magnitud y se establece el bit de signo a 0 si el número es positivo o a 1 si es negativo.

● Ejemplo 1.15

Representar en signo y magnitud, utilizando 8 bits, los siguientes números:

- a) -45 b) 54 c) -73

Solución

- a) Para representar el número -45 , se considera primero su valor absoluto, 45 , y se convierte al sistema binario:

$$(45)_{10} = (101101)_2$$

Como se dispone de 7 bits para representar la magnitud, se añade un cero por la izquierda:

$$0101101$$

Como el número que se desea representar es negativo, el bit de signo debe ser 1. Por tanto,

$$-45 \rightarrow 10101101$$

b) El número 54 es positivo. Su representación en el sistema binario es

$$(54)_{10} = (110110)_2$$

Como se dispone de 7 bits para representar la magnitud, se añade un cero por la izquierda:

$$0110110$$

Como el número es positivo, el bit de signo debe ser 0. Por tanto,

$$54 \rightarrow 00110110$$

c) Para representar el número -73 , se considera primero su valor absoluto, 73, y se convierte al sistema binario:

$$(73)_{10} = (1001001)_2$$

Los 7 bits obtenidos constituyen directamente la magnitud. Como el número que se desea representar es negativo, el bit de signo debe ser 1. Por tanto,

$$-73 \rightarrow 11001001$$

●

El proceso inverso permite obtener el valor decimal correspondiente a una representación en signo y magnitud. Para ello, se interpreta por separado el bit de signo y los bits que representan la magnitud. El bit de signo determina si el número es positivo o negativo, mientras que los restantes bits se interpretan como un número binario sin signo para obtener su valor absoluto.

● Ejemplo 1.16

Determinar el valor decimal de los siguientes números representados en signo y magnitud utilizando 8 bits:

a) 01011010

b) 10110110

c) 11001011

Solución

a) El bit de signo es 0, por lo que el número es positivo. Los siete bits restantes representan la magnitud:

$$1011010$$

Su valor decimal es

$$1 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 64 + 16 + 8 + 2 = 90$$

Por tanto,

$$01011010 \rightarrow 90$$

- b) El bit de signo es 1, por lo que el número es negativo. Los siete bits restantes representan la magnitud:

$$0110110$$

Su valor decimal es

$$0 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 32 + 16 + 4 + 2 = 54$$

Por tanto,

$$10110110 \rightarrow -54$$

- c) El bit de signo es 1, por lo que el número es negativo. Los siete bits restantes representan la magnitud:

$$1001011$$

Su valor decimal es

$$1 \cdot 2^6 + 0 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 64 + 8 + 2 + 1 = 75$$

Por tanto,

$$11001011 \rightarrow -75$$

Complemento a 1

Otro sistema utilizado para representar números enteros positivos y negativos es la representación en complemento a 1. Para definir este sistema de representación es necesario introducir previamente la operación de complementación de una secuencia binaria.

Dada una secuencia binaria A , se denomina **complemento** de A , y se representa por $\bar{A}$, a la secuencia binaria que se obtiene sustituyendo cada bit 0 por 1 y cada bit 1 por 0.

Por ejemplo, si

$$A = 01101001$$

su complemento es

$$\bar{A} = 10010110$$

La **representación en complemento a 1** es un sistema de representación que permite representar números enteros positivos y negativos. Los números positivos se representan directamente en el sistema binario utilizando el número de bits establecido.

Para obtener la representación en complemento a 1 de un número negativo se parte de la representación del correspondiente número positivo y se complementan todos sus bits.

Por ejemplo, utilizando 8 bits, el número +5 se representa como

$$+5 \rightarrow 00000101$$

Para obtener la representación de -5, se complementan todos los bits de la representación de +5:

$$\begin{array}{rcl} +5 & 00000101 \\ -5 & 11111010 \end{array}$$

Por tanto,

$$-5 \rightarrow 11111010$$

Como en signo y magnitud, en complemento a 1 el bit más significativo permite determinar el signo del número: si es 0, el número es positivo, y si es 1, el número es negativo.

Si se utilizan n bits, el mayor número positivo que puede representarse es

$$2^{n-1} - 1$$

y el menor número negativo es

$$-(2^{n-1} - 1)$$

Con n bits, la **representación en complemento a 1** permite representar los valores comprendidos entre

$$-(2^{n-1} - 1) \leq x \leq 2^{n-1} - 1 \quad (1.5)$$

Al igual que ocurre en signo y magnitud, en complemento a 1 existen dos representaciones del valor cero. La representación del cero positivo está formada por todos los bits a 0:

$$00 \dots 0$$

Al complementar todos sus bits se obtiene la representación del cero negativo:

$$11 \dots 1$$

Por tanto, aunque con n bits pueden formarse 2^n secuencias diferentes, dos de ellas representan el valor cero y solamente pueden representarse $2^n - 1$ valores enteros diferentes.

● Ejemplo 1.17

Se dispone de 6 bits para representar números enteros en complemento a 1. ¿Cuántos valores enteros diferentes pueden representarse? ¿Cuáles son el menor y el mayor valor representables?

Solución

Con 6 bits pueden formarse

$$2^6 = 64$$

secuencias diferentes. Sin embargo, en complemento a 1 existen dos representaciones del valor cero, por lo que solamente pueden representarse

$$64 - 1 = 63$$

valores enteros diferentes.

Como el mayor número positivo que puede representarse con 6 bits es

$$2^{6-1} - 1 = 31$$

los valores representables están comprendidos entre -31 y 31 .

Por tanto, con 6 bits pueden representarse 63 valores enteros diferentes, comprendidos entre -31 y 31 .

● Ejemplo 1.18

Representar en complemento a 1, utilizando 8 bits, los siguientes números:

- a) -37 b) 26 c) -83

Solución

- a) Para representar el número -37 , se considera primero su valor absoluto, 37 , y se convierte al sistema binario:

$$(37)_{10} = (100101)_2$$

Como la representación debe realizarse utilizando 8 bits, se añaden ceros por la izquierda hasta completar los 8 bits:

$$+37 \rightarrow 00100101$$

Como el número que se desea representar es negativo, se complementan todos los bits de la representación obtenida:

$$-37 \rightarrow 11011010$$

- b) El número 26 es positivo, por lo que se convierte directamente al sistema binario:

$$(26)_{10} = (11010)_2$$

y se añaden ceros por la izquierda hasta completar los 8 bits:

$$26 \rightarrow 00011010$$

- c) Para representar el número -83 , se considera primero su valor absoluto, 83 , y se convierte al sistema binario:

$$(83)_{10} = (1010011)_2$$

Como la representación debe realizarse utilizando 8 bits, se añade un cero por la izquierda:

$$+83 \rightarrow 01010011$$

Como el número que se desea representar es negativo, se complementan todos los bits de la representación obtenida:

$$-83 \rightarrow 10101100$$

●

El proceso inverso permite obtener el valor decimal correspondiente a un número representado en complemento a 1. Para ello, se examina en primer lugar el bit más significativo. Si este bit es 0, el número es positivo y su valor se obtiene interpretando directamente la secuencia como un número binario. Si el bit más significativo es 1, el número es negativo; en este caso, se complementan todos los bits para obtener la representación de su valor absoluto y posteriormente se asigna signo negativo al resultado.

● Ejemplo 1.19

Determinar el valor decimal de los siguientes números representados en complemento a 1 utilizando 8 bits:

- a) 01011010 b) 11001011 c) 10110100

Solución

- a) El bit más significativo es 0, por lo que el número es positivo. Su valor se obtiene interpretando directamente la secuencia como un número binario:

$$0 \cdot 2^7 + 1 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 64 + 16 + 8 + 2 = 90$$

Por tanto,

$$01011010 \rightarrow 90$$

- b) El bit más significativo es 1, por lo que el número es negativo. Para obtener su valor absoluto se complementan todos los bits:

$$1\ 1\ 0\ 0\ 1\ 0\ 1\ 1 \longrightarrow 0\ 0\ 1\ 1\ 0\ 1\ 0\ 0$$

El valor decimal de la secuencia obtenida es

$$1 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^2 = 32 + 16 + 4 = 52$$

Por tanto,

$$1\ 1\ 0\ 0\ 1\ 0\ 1\ 1 \longrightarrow -52$$

- c) El bit más significativo es 1, por lo que el número es negativo. Para obtener su valor absoluto se complementan todos los bits:

$$1\ 0\ 1\ 1\ 0\ 1\ 0\ 0 \longrightarrow 0\ 1\ 0\ 0\ 1\ 0\ 1\ 1$$

El valor decimal de la secuencia obtenida es

$$1 \cdot 2^6 + 1 \cdot 2^3 + 1 \cdot 2^1 + 1 \cdot 2^0 = 64 + 8 + 2 + 1 = 75$$

Por tanto,

$$1\ 0\ 1\ 1\ 0\ 1\ 0\ 0 \longrightarrow -75$$

●

Complemento a 2

La representación en complemento a 2 utiliza la suma de números binarios. Por este motivo, antes de definir este sistema de representación es necesario estudiar brevemente cómo se realiza esta operación.

La **suma binaria** se realiza de forma análoga a la suma de números decimales, comenzando por los bits situados más a la derecha y avanzando hacia la izquierda. Las reglas básicas para sumar dos bits son

$$\begin{array}{ll} 0 + 0 = 0 & 0 + 1 = 1 \\ 1 + 0 = 1 & 1 + 1 = 10 \end{array}$$

En la última suma, $1 + 1 = 2$, pero el número 2 se representa en el sistema binario como $(10)_2$. Por tanto,

$$1 + 1 = 10$$

en binario. Esto indica que se escribe un 0 en la posición correspondiente y se genera un **bit de acarreo** (*carry*) de valor 1, que se añade a la suma de la posición situada inmediatamente a su izquierda.

Cuando existe un acarreo procedente de la posición anterior, este debe incluirse en la suma. En particular, $1 + 1 + 1 = 3$, pero el número 3 se representa en el sistema binario como $(11)_2$. Por tanto,

$$1 + 1 + 1 = 11$$

en binario. Esto indica que se escribe un 1 en la posición correspondiente y se genera nuevamente un acarreo de 1 hacia la posición situada inmediatamente a su izquierda.

● Ejemplo 1.20

Realizar la siguiente suma de números binarios:

$$(101101)_2 + (011011)_2$$

Solución

Se colocan los dos números uno debajo del otro y se realiza la suma comenzando por los bits situados más a la derecha:

$$\begin{array}{r} 101101 \\ + 011011 \\ \hline \end{array}$$

En la primera posición se tiene $1 + 1 = 10$. Por tanto, se escribe un 0 y se genera un acarreo de 1:

$$\begin{array}{r} 1 \\ 101101 \\ + 011011 \\ \hline 0 \end{array}$$

En la segunda posición se suman los dos bits y el acarreo,

$$0 + 1 + 1 = 10$$

por lo que se escribe otro 0 y se genera nuevamente un acarreo de 1:

$$\begin{array}{r} 1 \\ 101101 \\ + 011011 \\ \hline 00 \end{array}$$

En la tercera posición,

$$1 + 0 + 1 = 10$$

por lo que se escribe un 0 y se genera un nuevo acarreo de 1:

$$\begin{array}{r} 1 \\ 101101 \\ + 011011 \\ \hline 000 \end{array}$$

En la cuarta posición,

$$1 + 1 + 1 = 11$$

por lo que se escribe un 1 y se genera un acarreo de 1:

$$\begin{array}{r} 1 \\ 101101 \\ + 011011 \\ \hline 1000 \end{array}$$

En la quinta posición,

$$0 + 1 + 1 = 10$$

por lo que se escribe un 0 y se genera nuevamente un acarreo de 1:

$$\begin{array}{r} 1 \\ 101101 \\ + 011011 \\ \hline 01000 \end{array}$$

Finalmente, en la sexta posición,

$$1 + 0 + 1 = 10$$

por lo que se escribe un 0 y el acarreo de 1 pasa a constituir un nuevo bit situado a la izquierda:

$$\begin{array}{r} 101101 \\ + 011011 \\ \hline 1001000 \end{array}$$

Por tanto,

$$(101101)_2 + (011011)_2 = (1001000)_2$$

Puede comprobarse el resultado convirtiendo los números al sistema decimal:

$$(101101)_2 = 45 \quad (011011)_2 = 27$$

y

$$45 + 27 = 72 = (1001000)_2$$

Cuando se trabaja con un número fijo de bits, una suma puede producir un bit de acarreo más allá de la posición más significativa. Este bit recibe el nombre de **acarreo de salida** (*carry out*).

Por ejemplo, si se suman utilizando 4 bits los números

$$1101 \quad \text{y} \quad 0111$$

se obtiene

$$\begin{array}{r} 1101 \\ + 0111 \\ \hline 10100 \end{array}$$

El resultado completo necesita 5 bits. Si la operación se realiza utilizando únicamente 4 bits, el acarreo de salida situado a la izquierda queda fuera de la representación y se conservan solamente los 4 bits menos significativos:

$$10100 \rightarrow 0100$$

Esta circunstancia será importante al trabajar con números representados utilizando un número fijo de bits.

Una vez explicado cómo se realiza la suma de números binarios, se dispone de los conocimientos necesarios para estudiar la representación de números enteros en complemento a 2.

La **representación en complemento a 2** es un sistema de representación que permite representar números enteros positivos y negativos. Al igual que en complemento a 1, los números positivos se representan directamente en el sistema binario utilizando el número de bits establecido.

Para obtener la representación en complemento a 2 de un número negativo se parte de la representación del correspondiente número positivo, se complementan todos sus bits y se suma 1 al resultado.

Por ejemplo, utilizando 8 bits, el número +5 se representa como

$$+5 \rightarrow 00000101$$

Al complementar todos sus bits se obtiene

$$11111010$$

y, sumando 1,

$$\begin{array}{r} 11111010 \\ + 00000001 \\ \hline 11111011 \end{array}$$

Por tanto,

$$-5 \rightarrow 11111011$$

Como en los sistemas anteriores para representar números con signo, en complemento a 2 el bit más significativo permite determinar el signo del número: si es 0, el número es no negativo, mientras que si es 1, el número es negativo.

A diferencia de las representaciones en signo y magnitud y en complemento a 1, en complemento a 2 el valor cero tiene una única representación:

$$00\dots 0$$

La secuencia formada por un 1 seguido de $n - 1$ ceros no representa un cero negativo, sino el menor número que puede representarse con n bits:

$$\underbrace{10\dots 0}_{n \text{ bits}} \rightarrow -2^{n-1}$$

Por otra parte, el mayor número positivo que puede representarse con n bits es

$$2^{n-1} - 1$$

Por tanto, en complemento a 2 el intervalo de valores representables no es simétrico respecto de cero: puede representarse un número negativo más que números positivos.

Con n bits, la **representación en complemento a 2** permite representar los valores comprendidos entre

$$-2^{n-1} \leq x \leq 2^{n-1} - 1 \quad (1.6)$$

Por ejemplo, con 8 bits pueden representarse los valores comprendidos entre

$$-2^7 = -128 \quad \text{y} \quad 2^7 - 1 = 127$$

es decir, un total de

$$2^8 = 256$$

valores enteros diferentes.

● Ejemplo 1.21

Se dispone de 6 bits para representar números enteros en complemento a 2. ¿Cuántos valores enteros diferentes pueden representarse? ¿Cuáles son el menor y el mayor valor representables?

Solución

Con 6 bits pueden formarse

$$2^6 = 64$$

secuencias diferentes. Como en complemento a 2 existe una única representación del valor cero, las 64 secuencias representan 64 valores enteros diferentes.

El menor valor representable es

$$-2^{6-1} = -32$$

y el mayor es

$$2^{6-1} - 1 = 31$$

Por tanto, con 6 bits pueden representarse 64 valores enteros diferentes, comprendidos entre -32 y 31 .

● Ejemplo 1.22

Representar en complemento a 2, utilizando 8 bits, los siguientes números:

- a) -37 b) 26 c) -83

Solución

- a) Para representar el número -37, se considera primero su valor absoluto, 37, y se convierte al sistema binario:

$$(37)_{10} = (100101)_2$$

Como la representación debe realizarse utilizando 8 bits, se añaden ceros por la izquierda hasta completar los 8 bits:

$$+37 \rightarrow 00100101$$

A continuación, se complementan todos sus bits:

$$00100101 \rightarrow 11011010$$

y se suma 1:

$$\begin{array}{r} 11011010 \\ + 00000001 \\ \hline 11011011 \end{array}$$

Por tanto,

$$-37 \rightarrow 11011011$$

- b) El número 26 es positivo, por lo que se convierte directamente al sistema binario:

$$(26)_{10} = (11010)_2$$

y se añaden ceros por la izquierda hasta completar los 8 bits:

$$26 \rightarrow 00011010$$

- c) Para representar el número -83, se considera primero su valor absoluto, 83, y se convierte al sistema binario:

$$(83)_{10} = (1010011)_2$$

Como la representación debe realizarse utilizando 8 bits, se añade un cero por la izquierda:

$$+83 \rightarrow 01010011$$

A continuación, se complementan todos sus bits:

$$01010011 \rightarrow 10101100$$

y se suma 1:

$$\begin{array}{r} 10101100 \\ + 00000001 \\ \hline 10101101 \end{array}$$

Por tanto,

$$-83 \rightarrow 10101101$$

El proceso inverso permite obtener el valor decimal correspondiente a un número representado en complemento a 2. Si el bit más significativo es 0, el número es no negativo y su valor se obtiene interpretando directamente la secuencia como un número binario.

Si el bit más significativo es 1, el número es negativo. Para obtener su valor absoluto se complementan todos los bits y se suma 1 al resultado. La secuencia obtenida se interpreta entonces como un número binario y se asigna signo negativo a su valor.

● Ejemplo 1.23

Determinar el valor decimal de los siguientes números representados en complemento a 2 utilizando 8 bits:

- a) 01011010 b) 11001100 c) 10110100

Solución

- a) El bit más significativo es 0, por lo que el número es no negativo. Su valor se obtiene interpretando directamente la secuencia como un número binario:

$$0 \cdot 2^7 + 1 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 64 + 16 + 8 + 2 = 90$$

Por tanto,

$$01011010 \rightarrow 90$$

- b) El bit más significativo es 1, por lo que el número es negativo. Para obtener su valor absoluto se complementan todos los bits:

$$11001100 \rightarrow 00110011$$

y se suma 1:

$$\begin{array}{r} 00110011 \\ + 00000001 \\ \hline 00110100 \end{array}$$

La secuencia obtenida representa el valor

$$1 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^2 = 32 + 16 + 4 = 52$$

Por tanto,

$$11001100 \rightarrow -52$$

- c) El bit más significativo es 1, por lo que el número es negativo. Para obtener su valor absoluto se complementan todos los bits:

$$10110100 \rightarrow 01001011$$

y se suma 1:

$$\begin{array}{r} 01001011 \\ + 00000001 \\ \hline 01001100 \end{array}$$

La secuencia obtenida representa el valor

$$1 \cdot 2^6 + 1 \cdot 2^3 + 1 \cdot 2^2 = 64 + 8 + 4 = 76$$

Por tanto,

$$10110100 \rightarrow -76$$

La representación en complemento a 2 presenta dos ventajas importantes respecto de las representaciones en signo y magnitud y en complemento a 1: el valor cero tiene una única representación y las operaciones aritméticas con números positivos y negativos pueden realizarse utilizando directamente la suma binaria. Por este motivo, **el complemento a 2 es el sistema habitualmente utilizado para representar números enteros con signo en los computadores.**

Exceso a M

La **representación en exceso a M** es un sistema utilizado para representar números enteros mediante un número fijo de bits. En este sistema se utiliza una cantidad fija M , denominada **exceso** o **sesgo** (*bias*). Un número entero x se representa mediante el número binario sin signo correspondiente a

$$x + M \quad (1.7)$$

Recíprocamente, si una secuencia de bits representa como entero sin signo el valor N , el número entero representado es

$$x = N - M \quad (1.8)$$

Por ejemplo, en una representación en exceso a 127 utilizando 8 bits, para representar el número 5 se calcula

$$5 + 127 = 132$$

y se representa 132 como un número binario sin signo de 8 bits:

$$(132)_{10} = (10000100)_2$$

Por tanto,

$$5 \rightarrow 10000100$$

De forma análoga, para representar el número -5 se calcula

$$-5 + 127 = 122$$

y

$$(122)_{10} = (01111010)_2$$

por lo que

$$-5 \rightarrow 01111010$$

● Ejemplo 1.24

Realizar las siguientes conversiones utilizando una representación en exceso a 127 de 8 bits:

a) Representar el número -45

b) Determinar el valor decimal de 10110110

Solución

a) Para representar el número -45 en exceso a 127, se suma el exceso al valor del número:

$$-45 + 127 = 82$$

A continuación, se representa 82 como un número binario sin signo de 8 bits:

$$(82)_{10} = (01010010)_2$$

Por tanto,

$$-45 \rightarrow 01010010$$

b) Para determinar el valor decimal representado por

$$10110110$$

se interpreta primero la secuencia como un número binario sin signo:

$$(10110110)_2 = 182$$

y se resta el exceso:

$$182 - 127 = 55$$

Por tanto,

$$10110110 \rightarrow 55$$

El valor del exceso M depende del sistema de representación utilizado. Por ejemplo, en una representación de números enteros en exceso a 3, para representar el número 25 se calcula

$$25 + 3 = 28$$

por lo que, utilizando 5 bits,

$$25 \rightarrow 11100$$

► **Nota** La representación de números enteros en exceso a 3 no debe confundirse con el código BCD exceso a 3 que aparecía nombrado en la Sección 1.3.5. En una representación de números enteros en exceso a 3, se suma 3 al valor del número entero completo. En el código BCD exceso a 3, en cambio, se suma 3 independientemente a cada dígito decimal y cada resultado se codifica mediante 4 bits.

Si se utilizan n bits, el valor binario sin signo almacenado puede estar comprendido entre 0 y $2^n - 1$. Como en una representación en exceso a M se almacena el valor $x + M$, debe cumplirse

$$0 \leq x + M \leq 2^n - 1$$

y, por tanto,

$$-M \leq x \leq 2^n - 1 - M$$

Con n bits, una **representación en exceso a M** permite representar los valores comprendidos entre

$$-M \leq x \leq 2^n - 1 - M \quad (1.9)$$

Por ejemplo, con 8 bits y exceso a 127 pueden representarse los valores comprendidos entre

$$-127 \quad \text{y} \quad 2^8 - 1 - 127 = 128$$

es decir, un total de $2^8 = 256$ valores enteros diferentes.

La representación en exceso a M se utiliza, entre otras aplicaciones, para representar exponentes en sistemas de representación de números en coma flotante. El valor del exceso M depende del sistema de representación utilizado. Por ejemplo, el estándar IEEE 754 utiliza un exceso de 127 para representar el exponente en el formato de precisión simple y un exceso de 1023 en el formato de precisión doble, como se verá posteriormente.

1.3.8. Representación de números reales

Los sistemas de representación estudiados anteriormente permiten representar números enteros. Sin embargo, un computador también debe poder representar números que tienen parte fraccionaria. Para representar estos números mediante una cantidad finita de bits es necesario establecer cómo se utilizan los bits disponibles para representar tanto la parte entera como la parte fraccionaria del número.

Existen dos formas fundamentales de hacerlo: la **representación en coma fija**, en la que la posición de la coma se establece previamente y permanece fija, y la **representación en coma flotante**, en la que su posición puede variar mediante el uso de un exponente.

Coma fija

En la **representación en coma fija**, se establece previamente una posición fija para la coma que separa la parte entera de la parte fraccionaria del número. De este modo, un determinado número de bits se utiliza para representar la parte entera y los restantes para representar la parte fraccionaria.

La posición de la coma no se almacena junto con el número, sino que viene determinada por el formato de representación utilizado. Por tanto, una misma secuencia de bits puede representar valores diferentes dependiendo de la posición establecida para la coma.

Por ejemplo, supóngase que se utilizan 8 bits y se establece que los 4 bits situados a la izquierda de la coma corresponden a la parte entera y los 4 situados a la derecha de la coma a la parte fraccionaria. La secuencia

$$1\ 0\ 1\ 1\ 0\ 1\ 1\ 0$$

se interpreta entonces como

$$1011.0110$$

La parte entera representa el valor

$$(1011)_2 = 11$$

mientras que la parte fraccionaria representa

$$0 \cdot 2^{-1} + 1 \cdot 2^{-2} + 1 \cdot 2^{-3} + 0 \cdot 2^{-4} = \frac{1}{4} + \frac{1}{8} = \frac{3}{8}$$

Por tanto,

$$(1011.0110)_2 = 11 + \frac{3}{8} = 11.375$$

Así, para el formato considerado,

$$1\ 0\ 1\ 1\ 0\ 1\ 1\ 0 \longrightarrow 11.375$$

● Ejemplo 1.25

Considerando una representación en coma fija de 8 bits, en la que los 4 bits situados a la izquierda de la coma corresponden a la parte entera y los 4 situados a la derecha a la parte fraccionaria, realizar las siguientes conversiones:

- a) Determinar el valor decimal de 0 1 1 0 1 0 1 0 b) Representar el número decimal 9.625

Solución

- a) Para determinar el valor decimal representado por

$$0\ 1\ 1\ 0\ 1\ 0\ 1\ 0$$

se sitúa la coma en la posición establecida por el formato:

$$0110.1010$$

La parte entera representa

$$(0110)_2 = 6$$

mientras que la parte fraccionaria representa

$$1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} + 0 \cdot 2^{-4} = \frac{1}{2} + \frac{1}{8} = 0.625$$

Por tanto,

$$0110.1010 \longrightarrow 6.625$$

- b) Para representar el número decimal 9.625, se convierten por separado su parte entera y su parte fraccionaria. La parte entera es

$$(9)_{10} = (1001)_2$$

y la parte fraccionaria es

$$(0.625)_{10} = (0.101)_2$$

Como se dispone de 4 bits para la parte fraccionaria, se añade un cero a la derecha:

$$(0.625)_{10} = (0.1010)_2$$

Por tanto,

$$9.625 \longrightarrow 10011010$$

El número de bits destinados a la parte fraccionaria determina los valores fraccionarios que pueden representarse. Si se utilizan f bits para la parte fraccionaria, el bit situado más a la derecha tiene peso 2^{-f} , que corresponde a la menor diferencia posible entre dos valores consecutivos representables.

En los ejemplos anteriores se utilizan 4 bits para la parte fraccionaria. Por tanto, el menor incremento que puede representarse es

$$2^{-4} = \frac{1}{16} = 0.0625$$

Cada valor representable se obtiene sumando este incremento al valor representable anterior. Así, los primeros valores son

$$0 + 0.0625 = 0.0625$$

$$0.0625 + 0.0625 = 0.125$$

$$0.125 + 0.0625 = 0.1875$$

$$0.1875 + 0.0625 = 0.25$$

$$0.25 + 0.0625 = 0.3125$$

Por tanto, los valores representables comienzan de la forma

$$0, 0.0625, 0.125, 0.1875, 0.25, 0.3125, 0.375, 0.4375, 0.5, \dots$$

hasta alcanzar el mayor valor permitido por el número de bits disponibles.

Se denomina **resolución** de una representación en coma fija a la diferencia entre dos valores consecutivos que pueden representarse. Si se utilizan f bits para la parte fraccionaria, la resolución es

$$2^{-f}$$

(1.10)

El número de bits destinados a la parte entera determina, a su vez, el mayor valor que puede representarse. Si se utilizan n bits en total, de los cuales f corresponden a la parte fraccionaria, quedan $n - f$ bits para la parte entera.

Si se consideran únicamente números no negativos, el menor valor representable es 0. El mayor se obtiene cuando todos los bits tienen valor 1:

$$\underbrace{11 \dots 1}_{n-f \text{ bits}} . \underbrace{11 \dots 1}_{f \text{ bits}}$$

cuyo valor es

$$2^{n-f} - 2^{-f}$$

Con n bits, de los cuales f se utilizan para representar la parte fraccionaria, una representación en coma fija de números no negativos permite representar los valores comprendidos entre

$$0 \leq x \leq 2^{n-f} - 2^{-f} \quad (1.11)$$

con una resolución de 2^{-f} .

Por ejemplo, en el formato anterior se utilizan 8 bits, de los cuales 4 corresponden a la parte entera y 4 a la parte fraccionaria. Por tanto, pueden representarse los valores comprendidos entre

$$0 \quad \text{y} \quad 2^4 - 2^{-4} = 15.9375$$

con una resolución de

$$2^{-4} = 0.0625$$

● Ejemplo 1.26

Considerar una representación en coma fija de 8 bits, en la que los 5 bits situados a la izquierda de la coma corresponden a la parte entera y los 3 situados a la derecha a la parte fraccionaria. Determinar la resolución y el rango de valores que pueden representarse.

Solución

Como se utilizan 3 bits para representar la parte fraccionaria, la resolución es

$$2^{-3} = \frac{1}{8} = 0.125$$

Por tanto, dos valores representables consecutivos se diferencian en 0.125. Los primeros valores que pueden representarse son

$$0, 0.125, 0.25, 0.375, 0.5, 0.625, 0.75, 0.875, 1, \dots$$

Para determinar el rango de representación, se consideran el menor y el mayor valor que pueden representarse. Como se están representando únicamente números no negativos, el menor valor corresponde a todos los bits a 0:

$$00000.000 = 0$$

El mayor se obtiene cuando todos los bits tienen valor 1:

$$11111.111$$

Por tanto,

$$2^5 - 2^{-3} = 32 - 0.125 = 31.875$$

Así, el rango de valores que pueden representarse es

$$0 \leq x \leq 31.875$$

con una resolución de

$$0.125$$

●

Hasta ahora se han considerado únicamente números no negativos. Para representar también números negativos en coma fija puede utilizarse alguno de los sistemas de representación de enteros con signo estudiados anteriormente. En particular, puede utilizarse el complemento a 2, considerando que un número determinado de bits corresponde a la parte fraccionaria.

Si se utilizan n bits, de los cuales f corresponden a la parte fraccionaria, una forma sencilla de interpretar la representación consiste en considerar inicialmente los n bits como un número entero en complemento a 2 y dividir el valor obtenido por 2^f .

Por ejemplo, considérese una representación en coma fija de 8 bits con 4 bits para la parte fraccionaria. La secuencia

$$1\ 1\ 1\ 1\ 1\ 0\ 0\ 0$$

interpretada como un número entero en complemento a 2 representa el valor -8 . Como se utilizan 4 bits para la parte fraccionaria, su valor en coma fija es

$$-\frac{8}{2^4} = -\frac{2^3}{2^4} = -\frac{1}{2} = -0.5$$

Por tanto,

$$1111.1000 \rightarrow -0.5$$

En este caso, la resolución sigue estando determinada por el número de bits destinados a la parte fraccionaria y es

$$2^{-f}$$

Por otra parte, como un número entero de n bits en complemento a 2 puede tomar valores comprendidos entre

$$-2^{n-1} \quad \text{y} \quad 2^{n-1} - 1$$

y la utilización de f bits para la parte fraccionaria equivale a dividir estos valores por 2^f , los extremos del rango se obtienen dividiendo también por 2^f .

El menor valor representable es, por tanto,

$$-\frac{2^{n-1}}{2^f} = -2^{n-f-1}$$

mientras que el mayor valor representable es

$$\frac{2^{n-1} - 1}{2^f} = \frac{2^{n-1}}{2^f} - \frac{1}{2^f} = 2^{n-f-1} - 2^{-f}$$

Por tanto, el rango de representación es

$$-2^{n-f-1} \leq x \leq 2^{n-f-1} - 2^{-f}$$

Con n bits, de los cuales f se utilizan para representar la parte fraccionaria, una **representación en coma fija en complemento a 2** permite representar los valores comprendidos entre

$$-2^{n-f-1} \leq x \leq 2^{n-f-1} - 2^{-f} \quad (1.12)$$

con una resolución de

$$2^{-f} \quad (1.13)$$

Para el formato considerado anteriormente, se utilizan 8 bits, de los cuales 4 corresponden a la parte fraccionaria. Por tanto, el rango de representación es

$$-2^{8-4-1} \leq x \leq 2^{8-4-1} - 2^{-4}$$

es decir,

$$-8 \leq x \leq 7.9375$$

y la resolución es

$$2^{-4} = 0.0625$$

● Ejemplo 1.27

Considerar una representación en coma fija de 8 bits en complemento a 2, con 3 bits para la parte fraccionaria. Realizar las siguientes conversiones:

- Representar el número decimal -5.25
- Determinar el valor decimal representado por 1 1 0 1 0 1 0 0

Solución

- Para representar el número decimal -5.25 , se multiplica su valor por 2^3 , ya que se utilizan 3 bits para la parte fraccionaria:

$$-5.25 \cdot 2^3 = -42$$

A continuación, se representa el número -42 en complemento a 2 utilizando 8 bits. La representación binaria de 42 es

$$(42)_{10} = (00101010)_2$$

Se complementan todos sus bits:

$$00101010 \rightarrow 11010101$$

y se suma 1:

$$11010101 + 1 = 11010110$$

Por tanto, la representación en coma fija de -5.25 es

$$11010.110$$

es decir,

$$-5.25 \rightarrow 1\ 1\ 0\ 1\ 0\ 1\ 1\ 0$$

- Para determinar el valor decimal representado por

$$1\ 1\ 0\ 1\ 0\ 1\ 0\ 0$$

se interpreta inicialmente la secuencia completa como un número entero en complemento a 2. La secuencia

$$11010100$$

representa el número entero -44 .

Como se utilizan 3 bits para la parte fraccionaria, se divide este valor por 2^3 :

$$-\frac{44}{2^3} = -\frac{44}{8} = -5.5$$

Por tanto,

$$11010.100 \rightarrow -5.5$$

●

● Ejemplo 1.28

Considerar una representación en coma fija de 10 bits en complemento a 2, con 5 bits para la parte fraccionaria. Determinar la resolución y el rango de valores que pueden representarse.

Solución

Como se utilizan 5 bits para representar la parte fraccionaria, la resolución es

$$2^{-5} = \frac{1}{32} = 0.03125$$

Por tanto, dos valores representables consecutivos se diferencian en 0.03125.

Para determinar el rango de representación se utiliza

$$-2^{n-f-1} \leq x \leq 2^{n-f-1} - 2^{-f}$$

con $n = 10$ y $f = 5$. Por tanto,

$$-2^{10-5-1} \leq x \leq 2^{10-5-1} - 2^{-5}$$

es decir,

$$-2^4 \leq x \leq 2^4 - \frac{1}{32}$$

y, finalmente,

$$-16 \leq x \leq 15.96875$$

Por tanto, esta representación permite representar valores comprendidos entre -16 y 15.96875 , con una resolución de 0.03125.

●

Como puede observarse, para un número fijo de bits existe un compromiso entre el rango de valores que pueden representarse y la resolución. Cuantos más bits se destinan a la parte fraccionaria, menor es la resolución, es decir, menor es la diferencia entre dos valores representables consecutivos, pero se reduce el rango de valores que pueden representarse. Por el contrario, cuantos más bits se destinan a la parte entera, mayor es el rango de representación, pero mayor es también la diferencia entre dos valores representables consecutivos.

Coma flotante

La representación en coma fija estudiada anteriormente establece de antemano cuántos bits se destinan a la parte entera y cuántos a la parte fraccionaria. Como consecuencia, para un número fijo de bits existe un compromiso entre el rango de valores que pueden representarse y la diferencia entre dos valores representables consecutivos.

Una forma de ampliar considerablemente el rango de representación consiste en permitir que la posición de la coma pueda variar. Esta es la idea fundamental de la **representación en coma flotante**. La denominación **coma flotante** (*floating-point*) hace referencia precisamente a que, a diferencia de lo que ocurre en la representación en coma fija, la posición de la coma no permanece fija, sino que puede desplazarse mediante el exponente.

Para comprender este sistema de representación, puede recordarse la notación científica utilizada habitualmente en el sistema decimal. En ella, un número se expresa como el producto de una cantidad por una potencia de 10. Por ejemplo,

$$6250 = 6.25 \cdot 10^3$$

mientras que

$$0.00625 = 6.25 \cdot 10^{-3}$$

En ambos casos aparece la misma cantidad, 6.25, pero cambia el exponente de la potencia de 10. Este exponente determina la posición que ocupa la coma en el número representado.

La misma idea puede aplicarse al sistema binario, sustituyendo las potencias de 10 por potencias de 2. Por ejemplo,

$$(10110.1)_2 = (1.01101)_2 \cdot 2^4$$

ya que desplazar la coma cuatro posiciones hacia la izquierda se compensa multiplicando por 2^4 .

Del mismo modo,

$$(0.00101101)_2 = (1.01101)_2 \cdot 2^{-3}$$

En ambos casos, $(1.01101)_2$ constituye la parte que contiene los **dígitos significativos** del número, es decir, los dígitos que se conservan al expresar el número mediante una potencia de la base, mientras que el exponente de la potencia de 2 determina la posición de la coma.

En una representación en coma flotante es necesario almacenar información sobre el signo del número, sus dígitos significativos y el desplazamiento de la coma.

En la **representación en coma flotante**, un número se expresa mediante tres componentes fundamentales:

- El **signo**, representado por un bit s , indica si el número es positivo o negativo, de manera que

$$s = 0 \rightarrow \text{número positivo} \quad s = 1 \rightarrow \text{número negativo}$$

- El **significando** (*significand*), también denominado **mantisa**, representado por m , contiene los dígitos significativos del número.
- El **exponente**, representado por e , determina el desplazamiento de la coma mediante una potencia de 2.

De forma general, un número binario en coma flotante puede expresarse como

$$x = (-1)^s \cdot m \cdot 2^e \quad (1.14)$$

► **Nota** Aunque en algunos textos se utiliza el término *mantisa* para designar el significando de un número en coma flotante, el término más preciso es *significando*. La palabra *mantisa* procede del ámbito de los logaritmos, donde designa la parte fraccionaria de un logaritmo decimal, mientras que el significando de un número en coma flotante contiene sus dígitos significativos. Por este motivo, en lo sucesivo se utilizará el término *significando*.

Por ejemplo, en la representación

$$(10110.1)_2 = (1.01101)_2 \cdot 2^4$$

el signo es positivo, por lo que $s = 0$, el significando es $m = (1.01101)_2$ y el exponente es $e = 4$. Por tanto,

$$(10110.1)_2 = (-1)^0 \cdot (1.01101)_2 \cdot 2^4$$

Si se considera ahora el número

$$(-10110.1)_2$$

el signo es negativo, por lo que $s = 1$, mientras que el significando y el exponente permanecen iguales. Por tanto,

$$(-10110.1)_2 = (-1)^1 \cdot (1.01101)_2 \cdot 2^4$$

En este caso,

$$s = 1, \quad m = (1.01101)_2, \quad e = 4$$

● Ejemplo 1.29

Expresar los siguientes números decimales en coma flotante en la forma

$$x = (-1)^s \cdot m \cdot 2^e$$

e identificar los valores de s , m y e :

a) $(13.25)_{10}$ b) $(-0.8125)_{10}$

Solución

a) En primer lugar, se convierte el número decimal 13.25 a binario. Su parte entera es

$$(13)_{10} = (1101)_2$$

mientras que su parte fraccionaria es

$$(0.25)_{10} = (0.01)_2$$

Por tanto,

$$(13.25)_{10} = (1101.01)_2$$

Una posible forma de expresar este número en coma flotante es

$$(1101.01)_2 = (11.0101)_2 \cdot 2^2$$

Como el número es positivo, $s = 0$. Por tanto,

$$(13.25)_{10} = (-1)^0 \cdot (11.0101)_2 \cdot 2^2$$

y sus componentes son

$$s = 0, \quad m = (11.0101)_2, \quad e = 2$$

b) En primer lugar, se convierte el valor absoluto del número decimal -0.8125 a binario:

$$(0.8125)_{10} = (0.1101)_2$$

Por tanto,

$$(-0.8125)_{10} = (-0.1101)_2$$

Una posible forma de expresar este número en coma flotante es

$$(-0.1101)_2 = (-1)^1 \cdot (0.1101)_2 \cdot 2^0$$

Como el número es negativo, $s = 1$. Por tanto, sus componentes son

$$s = 1, \quad m = (0.1101)_2, \quad e = 0$$

Un mismo número puede expresarse en coma flotante de diferentes formas. Por ejemplo,

$$(10110.1)_2 = (10.1101)_2 \cdot 2^3 = (1.01101)_2 \cdot 2^4$$

Para evitar esta multiplicidad de representaciones, se establece normalmente una forma determinada de escribir el significando.

Un número binario distinto de cero está expresado en **forma normalizada** cuando su significando tiene un único bit distinto de cero a la izquierda de la coma. Como en el sistema binario este bit necesariamente es 1, el número se expresa de la forma

$$x = (-1)^s \cdot (1.b_1b_2 \dots b_k)_2 \cdot 2^e \quad (1.15)$$

donde $(1.b_1b_2 \dots b_k)_2$ es el significando normalizado y cada b_i es un bit.

En el ejemplo anterior, la expresión

$$(10.1101)_2 \cdot 2^3$$

no está normalizada, ya que el significando tiene dos bits a la izquierda de la coma. En cambio,

$$(1.01101)_2 \cdot 2^4$$

sí está normalizada, puesto que su significando tiene un único bit distinto de cero a la izquierda de la coma. Por tanto, la representación normalizada del número es

$$(10110.1)_2 = (1.01101)_2 \cdot 2^4$$

y sus componentes son

$$s = 0, \quad m = (1.01101)_2, \quad e = 4$$

● Ejemplo 1.30

Expresar los siguientes números en forma normalizada e identificar los valores de s , m y e :

a) $(11010.101)_2$ b) $(-5.375)_{10}$

Solución

a) Para expresar el número

$$(11010.101)_2$$

en forma normalizada, se desplaza la coma cuatro posiciones hacia la izquierda:

$$(11010.101)_2 = (1.1010101)_2 \cdot 2^4$$

Como el número es positivo, $s = 0$. Por tanto,

$$(11010.101)_2 = (-1)^0 \cdot (1.1010101)_2 \cdot 2^4$$

y sus componentes son

$$s = 0, \quad m = (1.1010101)_2, \quad e = 4$$

b) Para expresar el número decimal -5.375 en forma normalizada, se convierte primero su valor absoluto a binario. La parte entera es

$$(5)_{10} = (101)_2$$

mientras que la parte fraccionaria es

$$(0.375)_{10} = (0.011)_2$$

Por tanto,

$$(-5.375)_{10} = (-101.011)_2$$

A continuación, se desplaza la coma dos posiciones hacia la izquierda:

$$(-101.011)_2 = (-1)^1 \cdot (1.01011)_2 \cdot 2^2$$

Por tanto, sus componentes son

$$s = 1, \quad m = (1.01011)_2, \quad e = 2$$

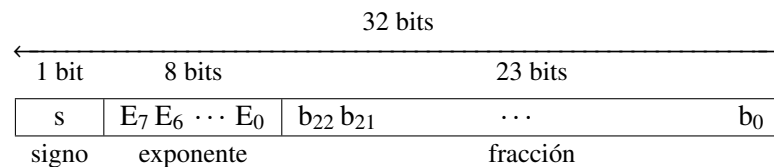


Hasta ahora se ha estudiado la forma de expresar un número en coma flotante sin especificar cómo se almacenan sus componentes mediante una cantidad determinada de bits. Para representar estos números en un computador es necesario establecer cuántos bits se destinan al signo, al exponente y al significando, así como la forma en que se codifica cada uno de ellos.

Uno de los sistemas más utilizados para la representación de números en coma flotante es el establecido por el **estándar IEEE 754**, que define diferentes formatos especificando el número de bits destinados al signo, al exponente y al significando, así como la forma de interpretar dichos bits., que define diferentes formatos especificando el número de bits destinados al signo, al exponente y al significando, así como la forma de interpretar dichos bits.

El estándar IEEE 754 define, entre otros, el formato de **precisión simple**, cuyas principales características son las siguientes:

- i) Utiliza 32 bits, distribuidos en tres campos: 1 bit para el signo, 8 bits para el exponente y 23 bits para la fracción. La estructura del formato es la siguiente:



- ii) Para los números normalizados, el significando tiene siempre la forma

$$m = (1.b_{22}b_{21} \dots b_0)_2$$

Por tanto, el primer bit del significando es siempre 1 y no es necesario almacenarlo. Este bit se denomina habitualmente **bit implícito**. Los 23 bits situados a la derecha de la coma constituyen la **fracción**, que es la parte del significando que se almacena explícitamente. De este modo, los 23 bits del campo de la fracción permiten disponer de 24 bits significativos al tener en cuenta el 1 implícito.

- iii) Para los números normalizados, el exponente se almacena en exceso a 127. Si e es el exponente del número normalizado y E es el valor entero sin signo representado por los 8 bits del campo del exponente, se cumple

$$E = e + 127 \tag{1.16}$$

y, por tanto,

$$e = E - 127$$

- iv) Para un número normalizado, su valor viene dado por

$$x = (-1)^s \cdot (1.b_{22}b_{21} \dots b_0)_2 \cdot 2^{E-127} \tag{1.17}$$

donde s es el bit de signo, E es el valor entero sin signo representado por los 8 bits del campo del exponente y $b_{22}b_{21} \dots b_0$ son los 23 bits del campo de la fracción.

● **Ejemplo 1.31**

Representar el número $(10110.1)_2$ en el formato IEEE 754 de precisión simple.

Solución

En primer lugar, se expresa el número en forma normalizada:

$$(10110.1)_2 = (1.01101)_2 \cdot 2^4$$

Como el número es positivo, el bit de signo es

$$s = 0$$

El exponente es $e = 4$, por lo que el valor que debe almacenarse en el campo del exponente es

$$E = e + 127 = 4 + 127 = 131$$

Este valor, expresado en binario mediante 8 bits, es

$$(131)_{10} = (10000011)_2$$

El significando normalizado es

$$m = (1.01101)_2$$

Como el 1 situado a la izquierda de la coma es implícito, únicamente se almacenan los bits situados a su derecha. El campo de la fracción se completa con ceros hasta alcanzar los 23 bits:

$$011010000000000000000000$$

Por tanto, la representación del número en el formato IEEE 754 de precisión simple es

signo	exponente	fracción
0	10000011	011010000000000000000000
1 bit	8 bits	23 bits

●

● **Ejemplo 1.32**

Representar el número $(-13.625)_{10}$ en el formato IEEE 754 de precisión simple.

Solución

En primer lugar, se convierte el valor absoluto del número a binario. Su parte entera es

$$(13)_{10} = (1101)_2$$

mientras que su parte fraccionaria es

$$(0.625)_{10} = (0.101)_2$$

Por tanto,

$$(-13.625)_{10} = (-1101.101)_2$$

A continuación, se expresa el número en forma normalizada:

$$(-1101.101)_2 = (-1)^1 \cdot (1.101101)_2 \cdot 2^3$$

Como el número es negativo, el bit de signo es

$$s = 1$$

El exponente es $e = 3$, por lo que el valor que debe almacenarse en el campo del exponente es

$$E = e + 127 = 3 + 127 = 130$$

Este valor, expresado en binario mediante 8 bits, es

$$(130)_{10} = (10000010)_2$$

El significando normalizado es

$$m = (1.101101)_2$$

Como el 1 situado a la izquierda de la coma es implícito, únicamente se almacenan los bits situados a su derecha. El campo de la fracción se completa con ceros hasta alcanzar los 23 bits:

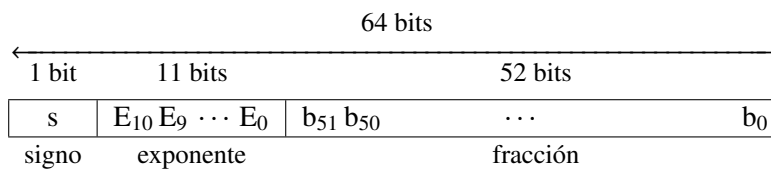
$$10110100000000000000000$$

Por tanto, la representación del número en el formato IEEE 754 de precisión simple es

signo	exponente	fracción
1	10000010	10110100000000000000000
1 bit	8 bits	23 bits

Además del formato de precisión simple, el estándar IEEE 754 define el formato de **precisión doble**, que utiliza **64 bits** para representar un número en coma flotante. Su funcionamiento es análogo al de la precisión simple, pero dispone de más bits para representar el exponente y la fracción. Esto permite ampliar el rango de valores representables y conservar más dígitos binarios significativos, consiguiendo así una mayor precisión en la representación.

En el **estándar IEEE 754 de precisión doble**, los 64 bits se distribuyen de la siguiente forma: 1 bit para el signo, 11 bits para el exponente y 52 bits para la fracción. La estructura del formato es la siguiente:



El exponente se almacena en exceso a 1023 y, al tener en cuenta el 1 implícito del significando normalizado, los 52 bits de la fracción permiten disponer de 53 bits significativos.

Por tanto, para los números normalizados se cumple

$$E = e + 1023 \quad (1.18)$$

y su valor viene dado por

$$x = (-1)^s \cdot (1.b_{51}b_{50} \dots b_0)_2 \cdot 2^{E-1023} \quad (1.19)$$

Las principales diferencias entre los formatos de precisión simple y precisión doble se resumen en la Tabla 1.7.

Hasta ahora se ha considerado la representación de números normalizados. Sin embargo, el estándar IEEE 754 reserva determinadas combinaciones de bits del campo del exponente para representar otros valores. En particular, cuando todos los bits del exponente son 0 o todos son 1, la representación recibe una interpretación especial.

Estas representaciones especiales son las siguientes:

Característica	Precisión simple	Precisión doble
Número total de bits	32	64
Bits de signo	1	1
Bits de exponente	8	11
Bits de fracción	23	52
Sesgo del exponente	127	1023
Bits significativos	24	53

Tabla 1.7 – Comparación de los formatos IEEE 754 de precisión simple y precisión doble

- Cuando el campo del exponente y el campo de la fracción tienen todos sus bits a 0, se representa el **cero**. Aunque matemáticamente existe un único valor cero, el estándar IEEE 754 admite dos representaciones para este valor: el **cero positivo** (+0), cuando el bit de signo es 0, y el **cero negativo** (−0), cuando el bit de signo es 1.

Estas dos representaciones permiten conservar información sobre el signo de determinados resultados que se aproximan a cero y mantener de forma coherente el signo en algunas operaciones. Por ejemplo, aunque la división por cero no está definida en los números reales, el estándar IEEE 754 establece que al dividir un número positivo finito distinto de cero entre +0 el resultado de la operación en coma flotante es $+\infty$, mientras que si se divide entre −0 el resultado es $-\infty$. Aunque +0 y −0 tienen representaciones diferentes, ambos corresponden al mismo valor numérico cero y se consideran iguales en una comparación numérica.

- Cuando todos los bits del exponente son 0, pero la fracción es distinta de cero, se representa un **número subnormal** (*subnormal number*). En este caso, el significando no tiene el 1 implícito utilizado en los números normalizados, sino que comienza por 0. Los números subnormales permiten representar valores de magnitud más próxima a cero que los números normalizados de menor magnitud.

Por ejemplo, en precisión simple el menor número normalizado positivo es

$$(1.000 \dots 0)_2 \cdot 2^{-126} = 2^{-126} \approx 1.1755 \cdot 10^{-38}$$

En cambio, el menor número subnormal positivo se obtiene cuando únicamente el último de los 23 bits de la fracción es 1:

$$(0.000 \dots 001)_2 \cdot 2^{-126} = 2^{-23} \cdot 2^{-126} = 2^{-149} \approx 1.4013 \cdot 10^{-45}$$

Por tanto, los números subnormales permiten representar números positivos mucho más próximos a cero que el menor número normalizado positivo. Su utilidad es evitar que los resultados cuya magnitud es menor que la del menor número normalizado tengan que representarse directamente como cero. De este modo, se produce una transición gradual desde los números normalizados de menor magnitud hasta cero, denominada **subdesbordamiento gradual** (*gradual underflow*).

- Cuando todos los bits del exponente son 1 y todos los bits de la fracción son 0, la representación corresponde a **infinito**. El bit de signo permite distinguir entre $+\infty$ y $-\infty$. Aunque la división por cero no está definida en los números reales, el estándar IEEE 754 establece que la división de un número finito distinto de cero por cero produce como resultado de la operación en coma flotante $+\infty$ o $-\infty$, dependiendo de los signos de los operandos.
- Cuando todos los bits del exponente son 1 y la fracción es distinta de cero, la representación corresponde a un **NaN** (*Not a Number*), que se utiliza para indicar el resultado de operaciones para las que no puede obtenerse un valor numérico definido. En la aritmética IEEE 754 se obtiene, por

ejemplo, al intentar realizar operaciones como $0/0$, $\infty - \infty$, $0 \cdot \infty$ o la raíz cuadrada de un número negativo cuando se trabaja únicamente con números reales.

► **Nota** Conviene recordar que $+\infty$ y $-\infty$ no son números reales. Como se estudia en Cálculo, estos símbolos pueden utilizarse para describir determinados comportamientos no acotados. En el estándar IEEE 754, además, se les asignan representaciones especiales que pueden aparecer, por ejemplo, cuando el resultado de una operación excede el rango de los números finitos representables. Por tanto, $+\infty$ y $-\infty$ no deben interpretarse como números reales ordinarios. ◀

Las diferentes interpretaciones de estas combinaciones especiales de bits se resumen en la Tabla 1.8.

Signo	Exponente	Fracción	Interpretación
0 o 1	Todos los bits a 0	Todos los bits a 0	$+0$ o -0
0 o 1	Todos los bits a 0	Distinta de 0	Número subnormal
0	Todos los bits a 1	Todos los bits a 0	$+\infty$
1	Todos los bits a 1	Todos los bits a 0	$-\infty$
0 o 1	Todos los bits a 1	Distinta de 0	NaN

Tabla 1.8 – Representaciones especiales del estándar IEEE 754